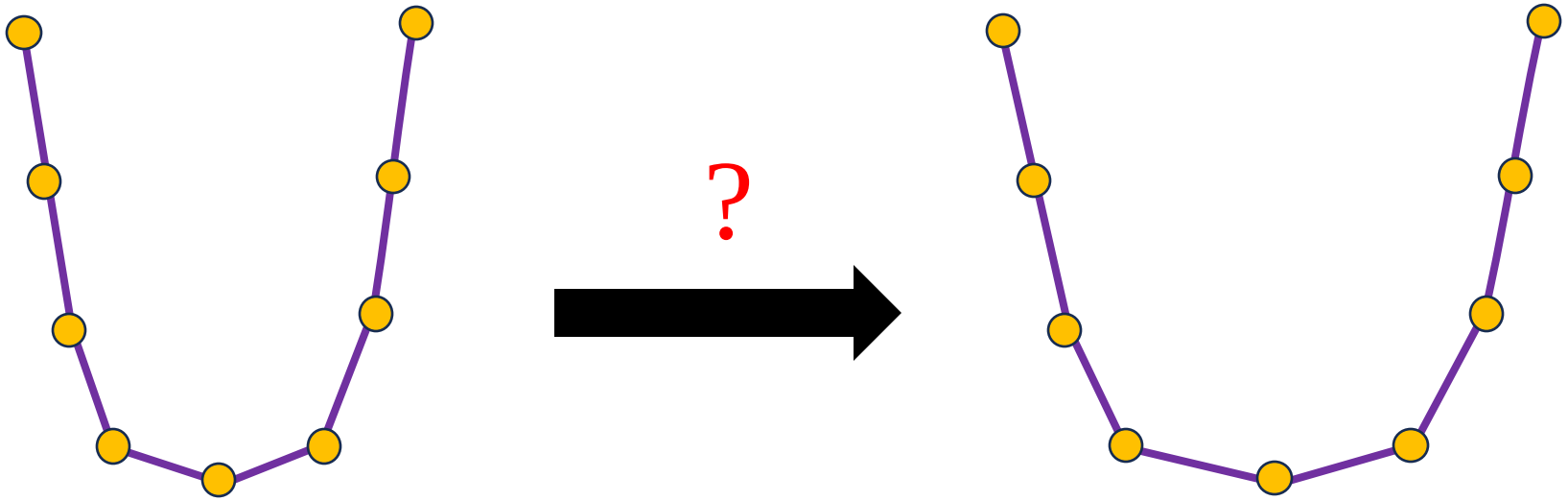
Splines

SNU Computer Science & Engineering

Some materials are stolen from the lecture slides of MIT Computer Graphics courses taught by Justin Solomon

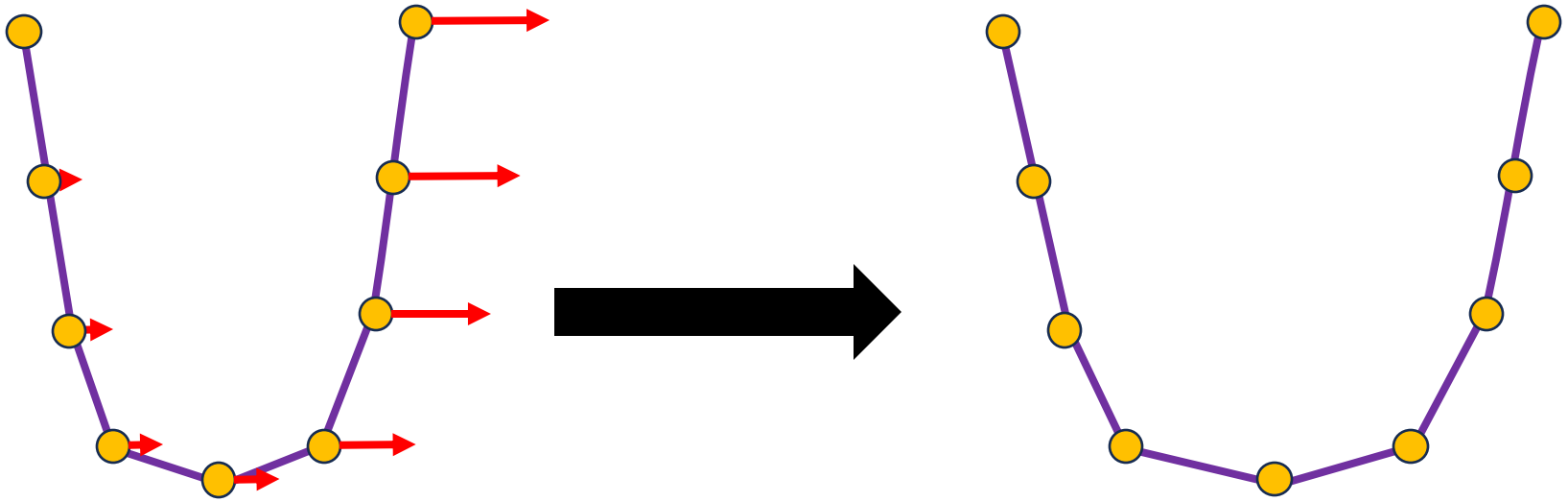
Motivation

- Let's say we have U-shaped curve made up of 8 line segments. What's the best way to make it a bit wider while keeping the initial position fixed?



Motivation

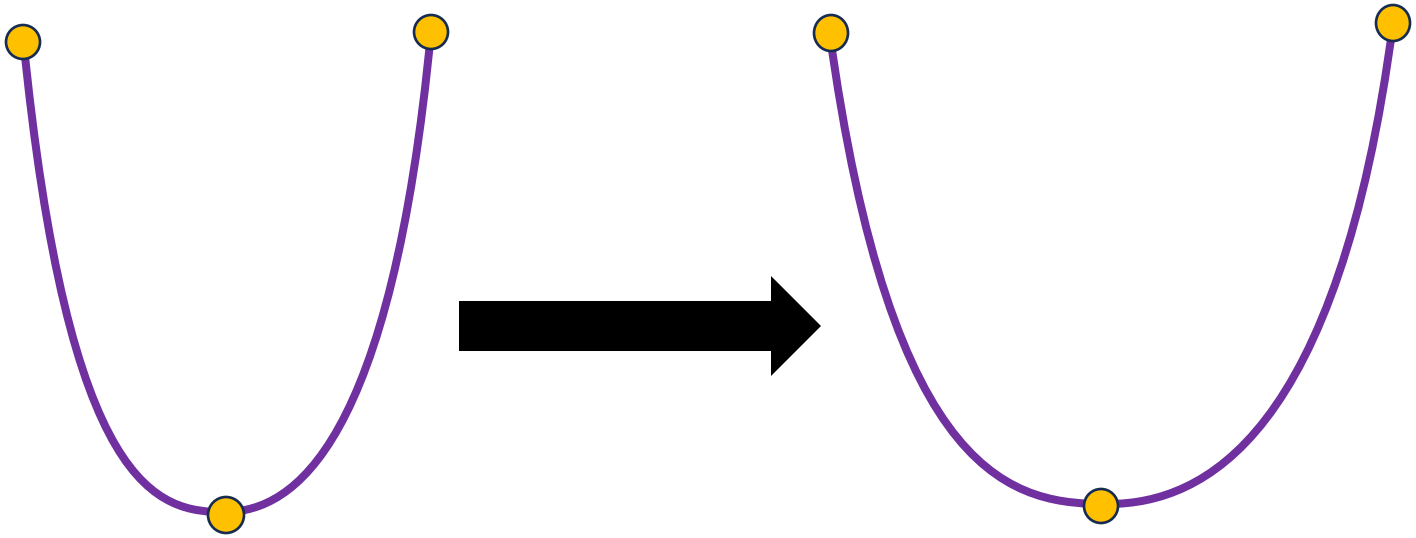
- A simplest solution would be:



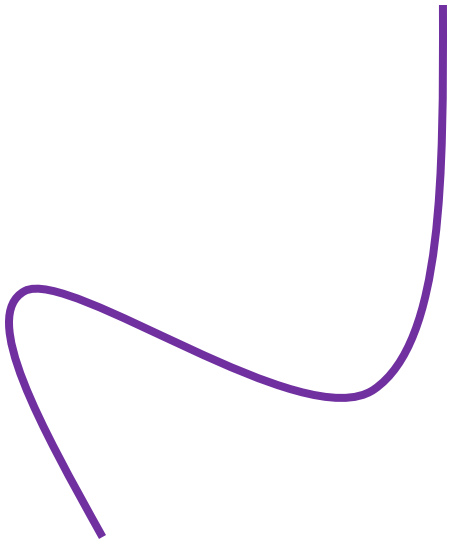
Is this method enough? What if we have 1000 segments?

Motivation

- We may want **higher-level representation** of curves and surfaces
- In other words, we would like to find a **smooth** curve/surface with a minimal number of **control points**

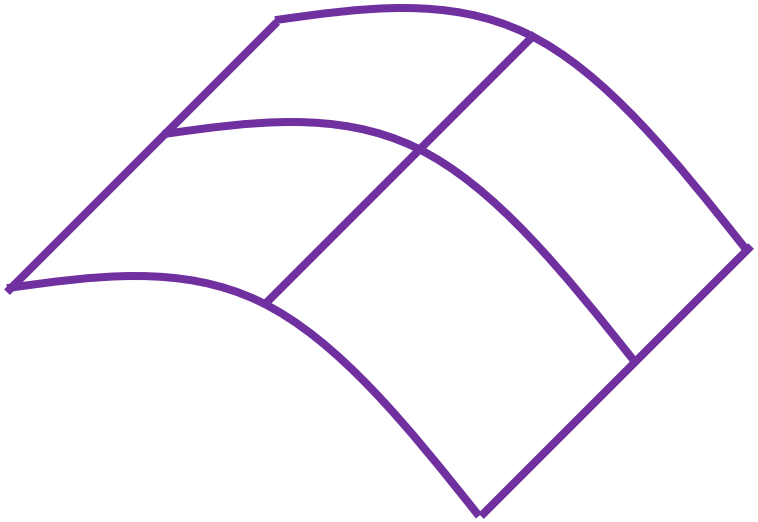


Parametric Geometry



$$\boldsymbol{p}(t) = (x(t), y(t))$$

$$\boldsymbol{p}\colon \mathbb{R} \to \mathbb{R}^2$$

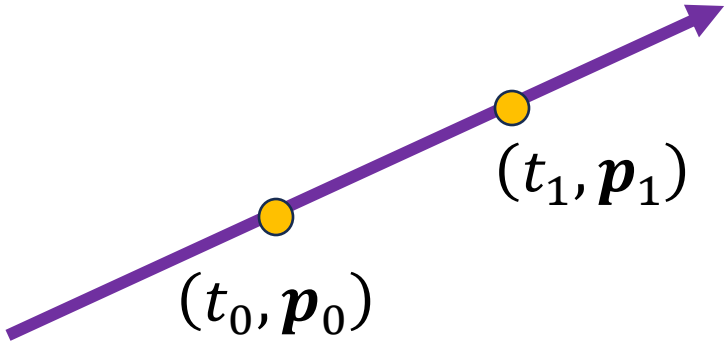


$$\boldsymbol{p}(u, v) = (x(u, v), y(u, v), z(u, v))$$

$$\boldsymbol{p}\colon \mathbb{R}^2 \to \mathbb{R}^3$$

Examples: Parametric Geometry

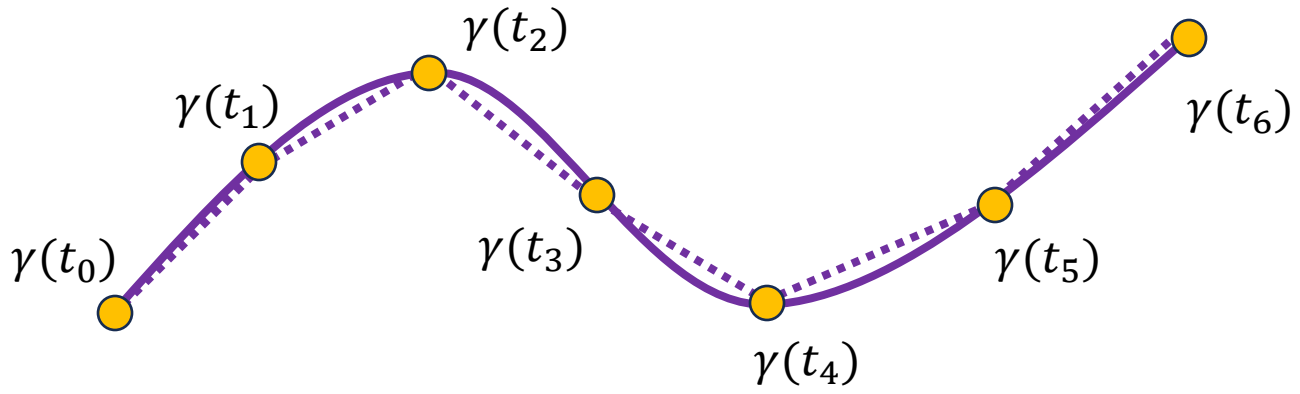
- Parameterized line



$$\boldsymbol{p}(t) = \frac{(t_1 - t)}{t_1 - t_0}\boldsymbol{p}_0 + \frac{(t - t_0)}{t_1 - t_0}\boldsymbol{p}_1$$

Why Parametric?

- It matches the intrinsic dimension of objects we want to manipulate although it may exist in higher dimensions
  - Curves (1D)
  - Surfaces (2D)
- Sampling process is easy, which is good for our graphics hardware to render (points, lines, triangles)

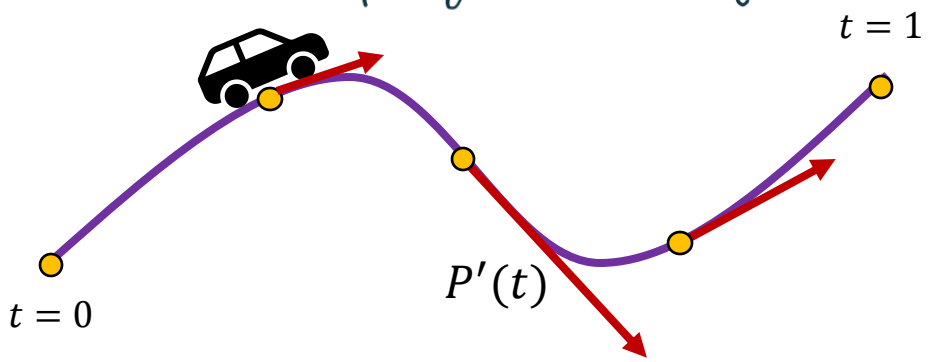


Differential Properties of Parametric Curves

- Velocity
- Tangent
- Curvature
- Normal

Velocity and Tangent

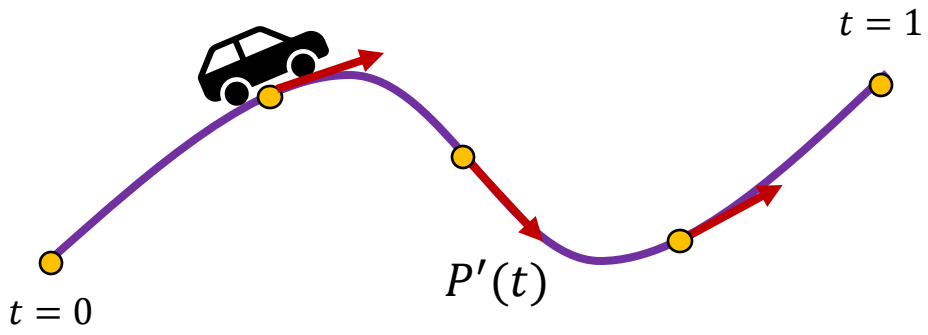
- Velocity
  - Instantaneous positional change, i.e., $P'(t)$
  - Can be easily understood by thinking t as time



Velocity and Tangent

- Tangent
  - The unit vector parallel to $P'(t)$
  - $T(t) = P'(t)/\|P'(t)\|$

normalized.

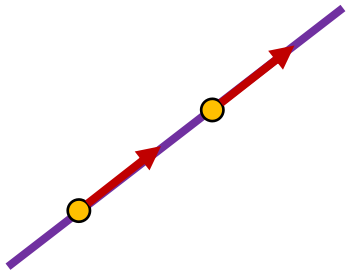


Curvature

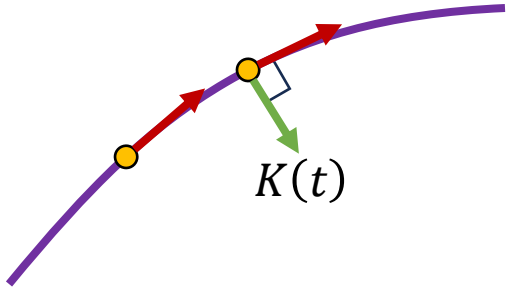
- Instantaneous tangential change

$$K(t) \propto T'(t)$$

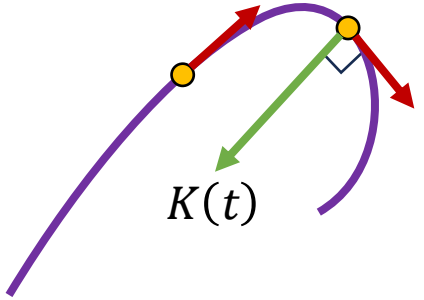
- Magnitude $\|K(t)\|$ is constant for a circle
- Always orthogonal to tangent i.e., $K(t) \cdot T(t) = 0$



Zero curvature



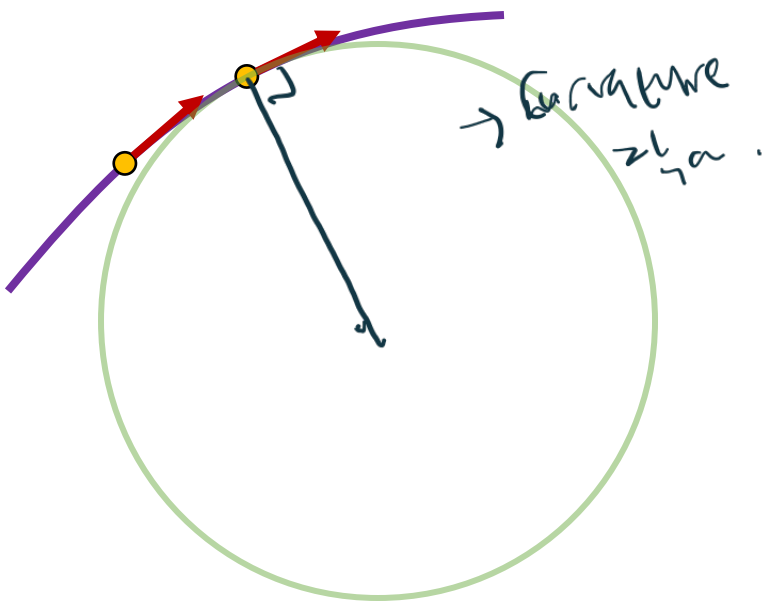
Low curvature



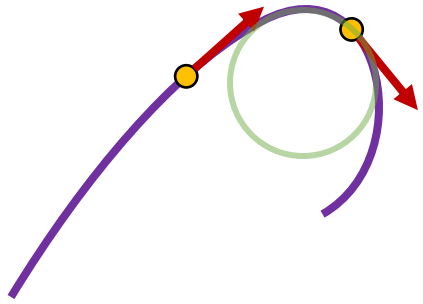
High curvature

Curvature: Geometric Interpretation

- When curves are parameterized by arc-length, $1/\|K(t)\|$ is the same as the radius of the circle that touches $P(t)$ at t



Low curvature

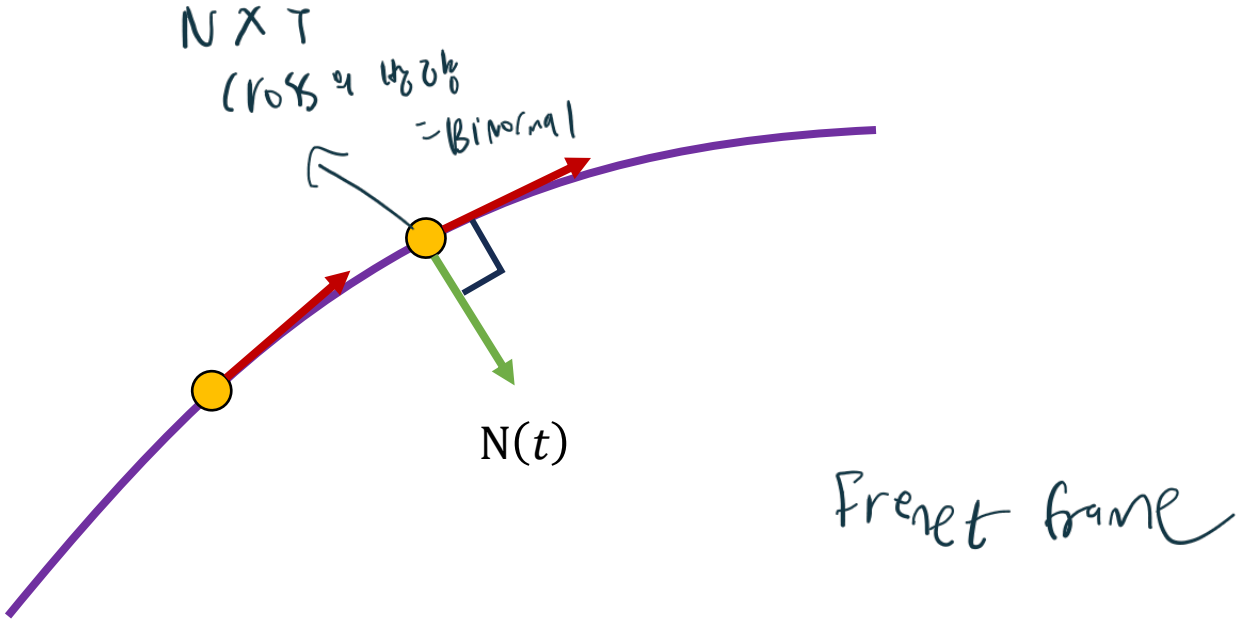


High curvature

Curve Normal

- Normalized curvature vector:

$$N(t) := K(t)/\|K(t)\| = T'(t)/\|T'(t)\|$$



Frenet frame

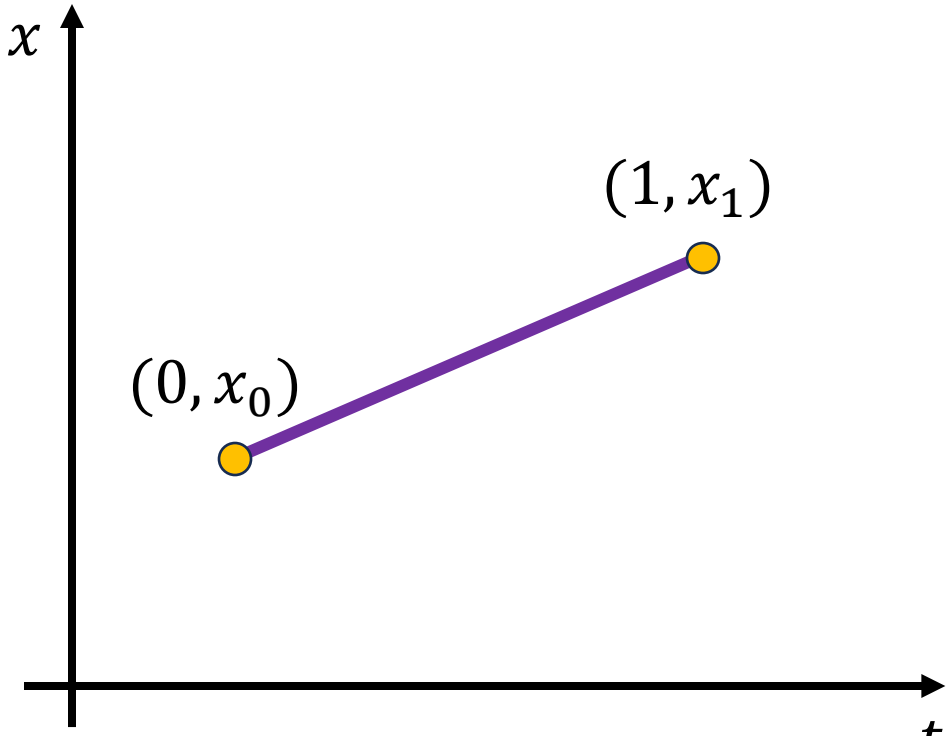
Agenda

1. Simple one-dimensional curves (functions)

2. Curves in higher dimensions

3. Complicated curves

1D Interpolation

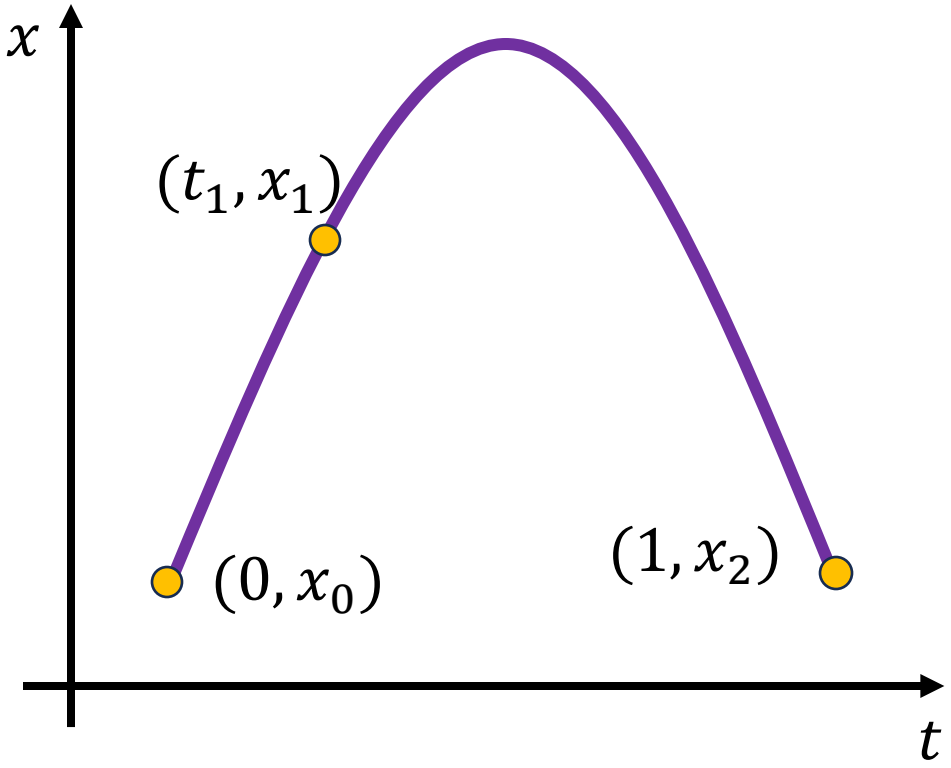


$$x(t) = at + b$$

$$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$$

$$\begin{bmatrix} b \\ a \end{bmatrix} = \begin{bmatrix} -1 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$$

1D Interpolation



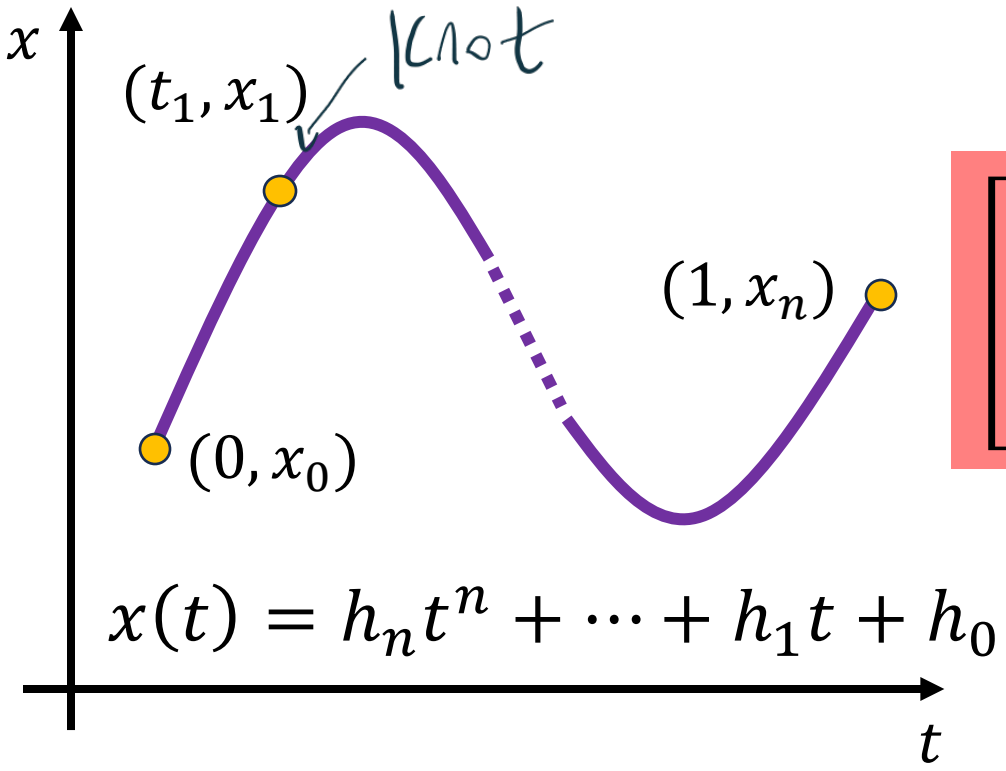
$$\begin{bmatrix} 0 & 0 & 1 \\ t_1^2 & t_1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \\ t_1^2 & t_1 & 1 \\ 1 & 1 & 1 \end{bmatrix}^{-1} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$

$$x(t) = at^2 + bt + c$$ ← order = 3 of polynomial

degree = 2

1D Interpolation



$$\begin{bmatrix} 1 & \cdots & 0 & 0 \\ 1 & \cdots & t_1^{n-1} & t_1^n \\ 1 & \ddots & \vdots & \vdots \\ 1 & \cdots & 1 & 1 \end{bmatrix} \begin{bmatrix} h_0 \\ h_1 \\ \vdots \\ h_n \end{bmatrix} = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$$

Do we really need to solve the linear system?

Lagrange Polynomial

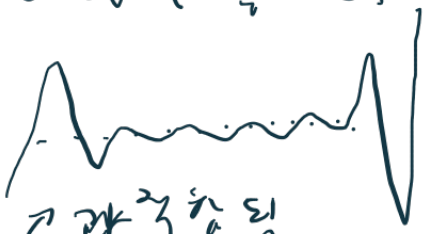
- Weighted sum of data points and cardinal functions

$$x(t) = L_0(t)x_0 + L_1(t)x_1 + \cdots + L_n(t)x_n$$

$$L_k(t) = \frac{(t-t_0)\cdots(t-t_{k-1})(t-t_{k+1})\cdots(t-t_n)}{(t_k-t_0)\cdots(t_k-t_{k-1})(t_k-t_{k+1})\cdots(t_k-t_n)}$$

- Cardinal polynomial functions

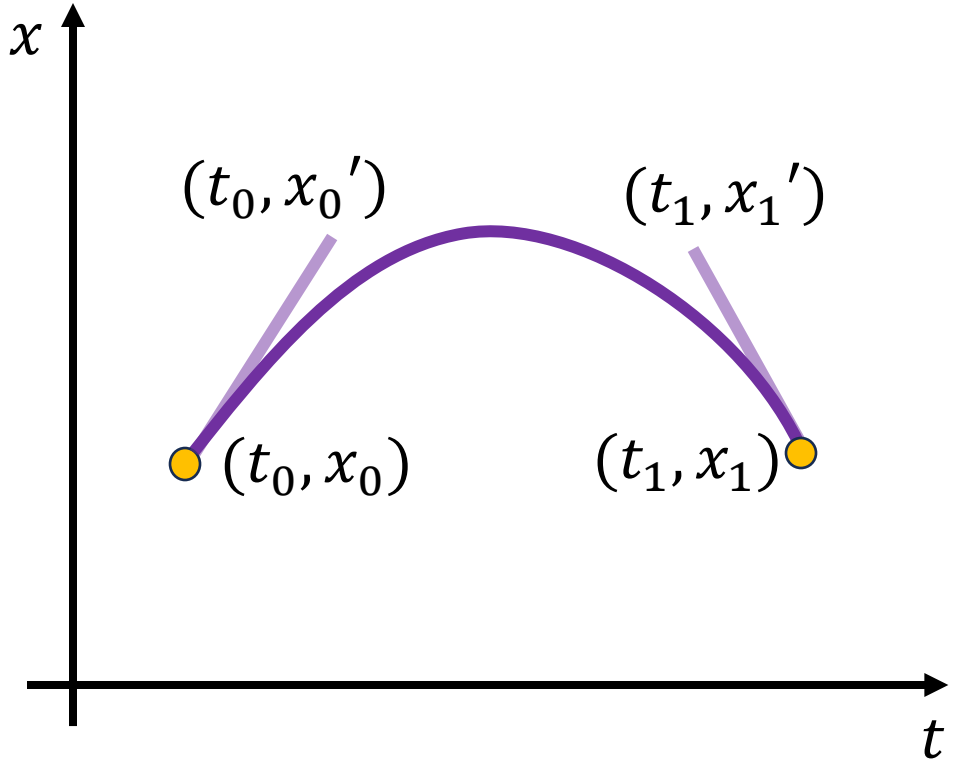
$$L_k(t_i) = \begin{cases} 1 & \text{if } k = i \\ 0 & \text{if } k \neq i \end{cases}$$



New Idea

tangent

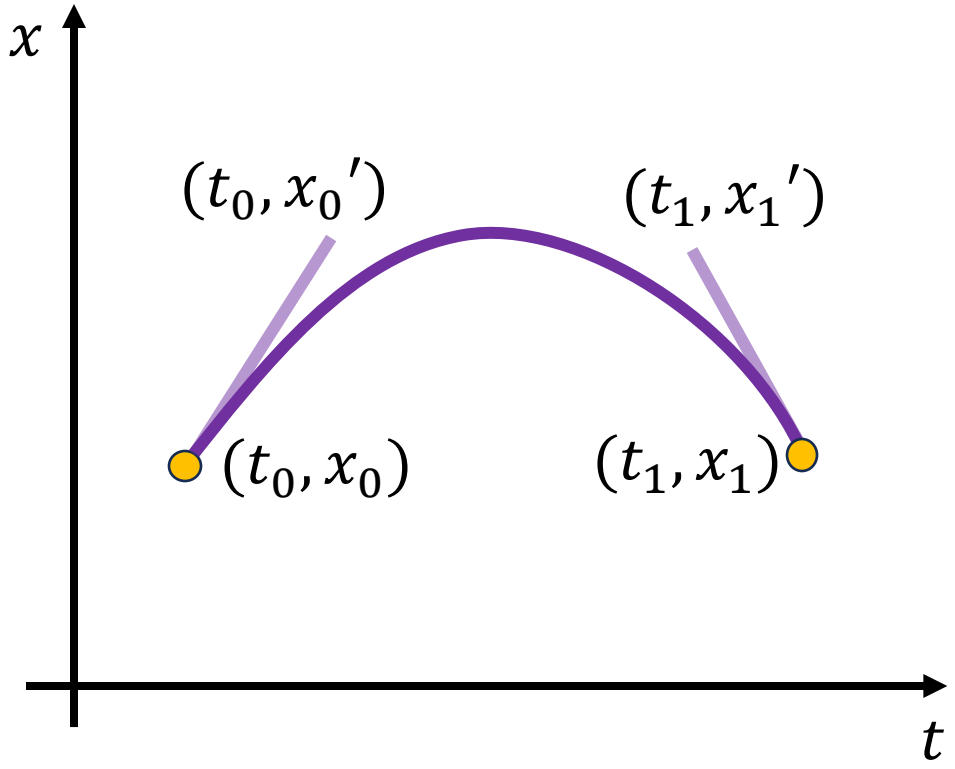
- We provide two control points and their derivatives at the end of the curve only, while intermediate parts are smoothly connected



How many constraints? What degree of polynomial is necessary?

4 constraint degree 3, order 4

Cubic Hermite Interpolation



$$x(t) = at^3 + bt^2 + ct + d$$
$$x'(t) = 3at^2 + 2bt + c$$

$$x(0) = d = x_0$$
$$x(1) = a + b + c + d = x_1$$
$$x'(0) = c = x_0'$$
$$x'(1) = 3a + 2b + c = x_1'$$

Cubic Hermite Interpolation

$$x(0) = d = x_0$$
$$x(1) = a + b + c + d = x_1$$
$$x'(0) = c = x'_0$$
$$x'(1) = 3a + 2b + c = x'_1$$

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 0 \\ 3 & 2 & 1 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} x_0 \\ x_1 \\ x'_0 \\ x'_1 \end{bmatrix}$$

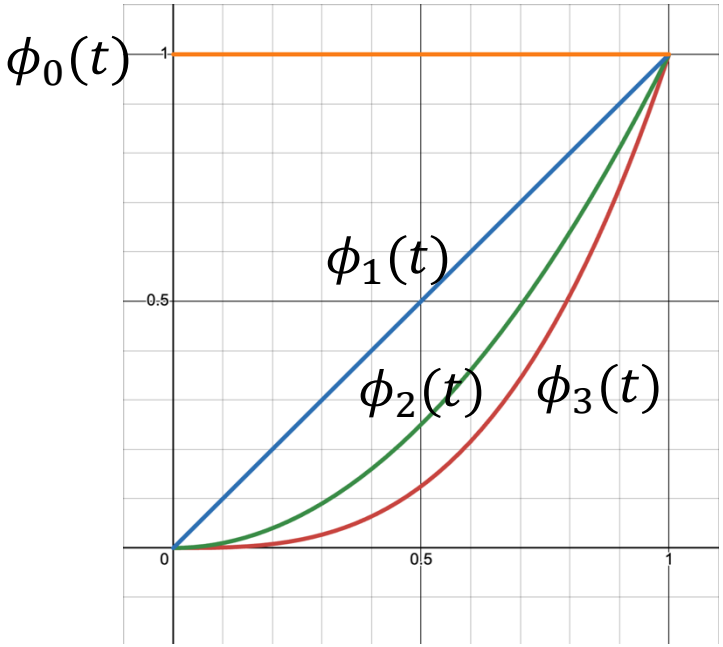
$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x'_0 \\ x'_1 \end{bmatrix}$$

↳ 이것은 식만으로 바로 얻을 수 있다.

Monomial Bases

- Given a cubic polynomial, we can think it as a 4D vector (a, b, c, d) of which bases are $t^3, t^2, t, 1$

Recall: functions can be vectors!



$$x(t) = at^3 + bt^2 + ct + d$$
$$= a\phi_3(t) + b\phi_2(t) + c\phi_1(t) + d\phi_0(t)$$

Change of Bases

- Other functions can also be bases of cubic polynomials, for example:

$$\underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & -1 & 1 \end{bmatrix}}_{\text{Change-of-basis matrix}} \underbrace{\begin{bmatrix} 1 \\ t \\ t^2 \\ t^3 \end{bmatrix}}_{\text{Monomial basis}} = \underbrace{\begin{bmatrix} 1 \\ 1+t \\ 1+t+t^2 \\ 1+t-t^2+t^3 \end{bmatrix}}_{\text{New basis}}$$

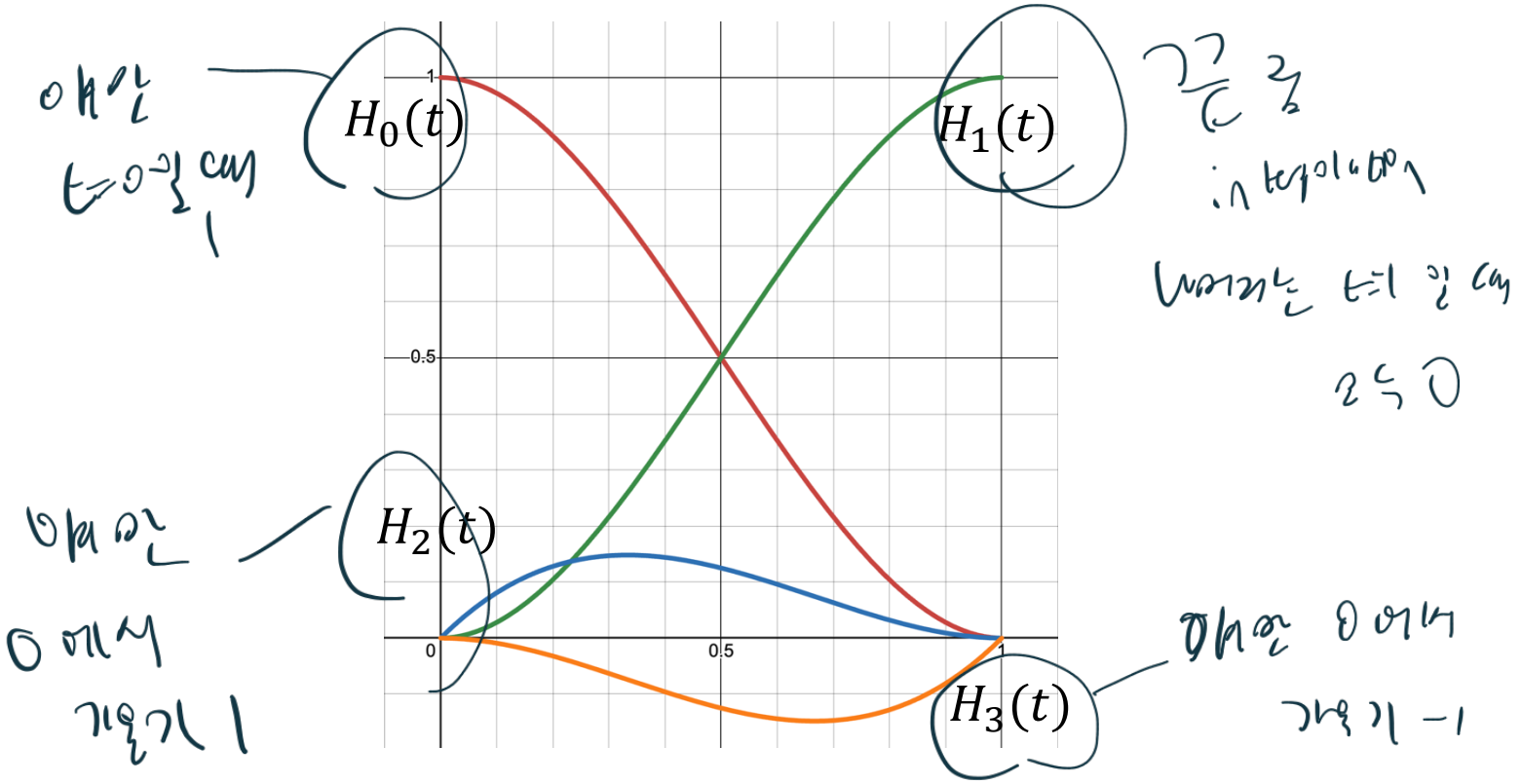
Cubic Hermite Interpolation

$$\begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix} = \begin{bmatrix} 2 & -2 & 1 & 1 \\ -3 & 3 & -2 & -1 \\ 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x'_0 \\ x'_1 \end{bmatrix}$$

$$\begin{aligned} x(t) &= at^3 + bt^2 + ct + d \\ &= (2x_0 - 2x_1 + x'_0 + x'_1)t^3 + \\ &\quad (-3x_0 + 3x_1 - 2x'_0 - x'_1)t^2 + x'_0 t + x_0 \end{aligned}$$

$$\begin{aligned} &= x_0(2t^3 - 3t^2 + 1) + x_1(-2t^3 + 3t^2) \\ &+ x'_0(t^3 - 2t^2 + t) + x'_1(t^3 - t^2) \end{aligned}$$

Hermite Bases



$$= x_0(2t^3 - 3t^2 + 1) + x_1(-2t^3 + 3t^2)$$
$$+x_0'(t^3 - 2t^2 + t) + x_1'(t^3 - t^2)$$

$$= x_0 H_0(t) + x_1 H_1(t) + x_0' H_2(t) + x_1' H_3(t)$$

Cubic Hermite Interpolation (Matrix Form)

$$x(t) = x_0(2t^3 - 3t^2 + 1) + x_1(-2t^3 + 3t^2) + x_0'(t^3 - 2t^2 + t) + x_1'(t^3 - t^2)$$

$$x(t) = \begin{bmatrix} x_0 & x_1 & x_0{}' & x_1' \end{bmatrix} \begin{bmatrix} 1 & 0 & -3 & 2 \\ 0 & 0 & 3 & -2 \\ 0 & 1 & -2 & 1 \\ 0 & 0 & -1 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ t \\ t^2 \\ t^3 \end{bmatrix}$$

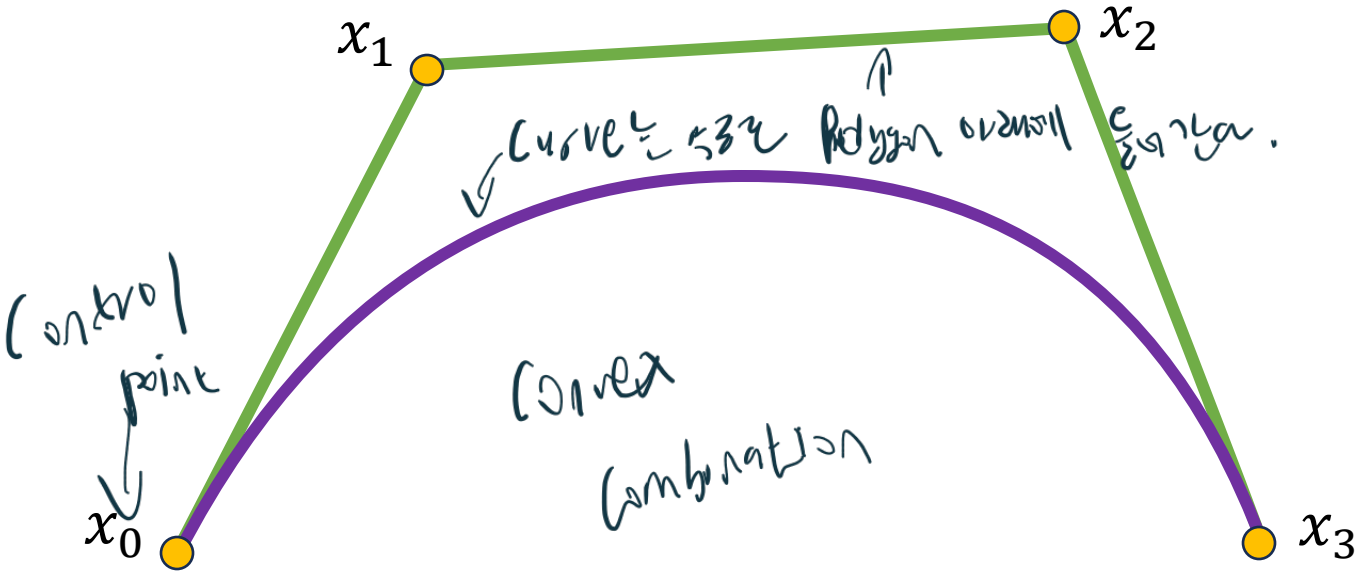
Geometry (control points) **G**

Change-of-basis **B**

Monomial basis **M**(t)

↳ hermite interpolation.

Cubic Bezier Curve
(a.k.a. Cubic Bernstein Polynomial)





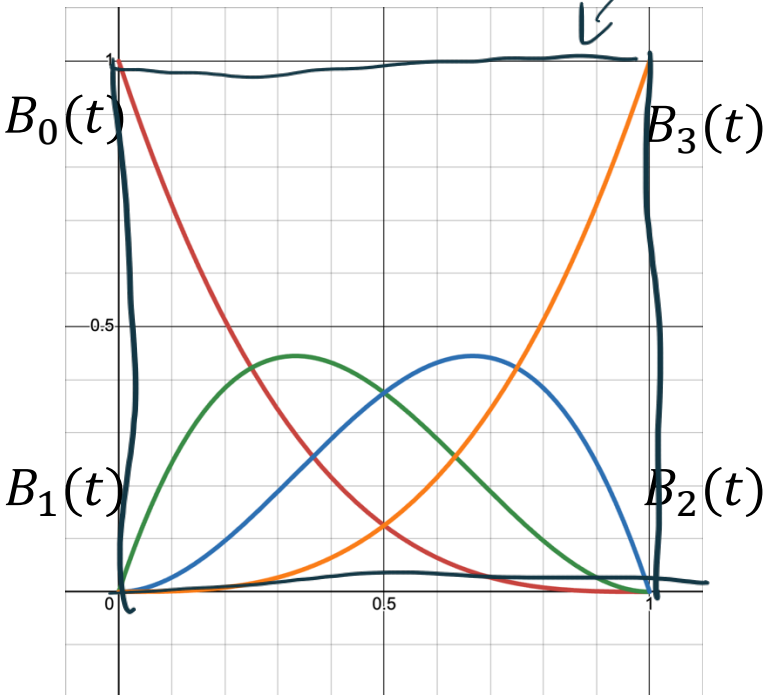
Pierre Bézier

$$x(t) = x_0 B_0(t) + x_1 B_1(t) + x_2 B_2(t) + x_3 B_3(t)$$

$$1 = \sum_{i=0}^{n} B_i(t) \quad \text{for any } 0 \leq t \leq 1$$

Partition of Unity

Bernstein Bases



$$B_i(t) = \binom{3}{i}(1-t)^{3-i}t^i$$

$$x(t) = x_0B_0(t) + x_1B_1(t) + x_2B_2(t) + x_3B_3(t)$$
$$= x_0(1-t)^3 + x_1 3t(1-t)^2 + x_2 3t^2(1-t) + x_3 t^3$$

Cubic Bezier Curve (Matrix Form)

$$x(t) = x_0(1-t)^3 + x_1 3t(1-t)^2 + x_2 3t^2(1-t) + x_3 t^3$$

$$x(t) = \begin{bmatrix} x_0 & x_1 & x_2 & x_3 \end{bmatrix} \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 0 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ t \\ t^2 \\ t^3 \end{bmatrix}$$

Geometry (control points) **G**

Change-of-basis **B**

Monomial basis **M**(t)

Properties of Cubic Bezier Curve

- End-point interpolation by diagonal proproperty of cubic blossoms

$$x_0 = x(0), \qquad x_3 = x(1)$$

- Three times the derivatives at the first and last control points are the same as the difference between the second-first and fourth-third control points

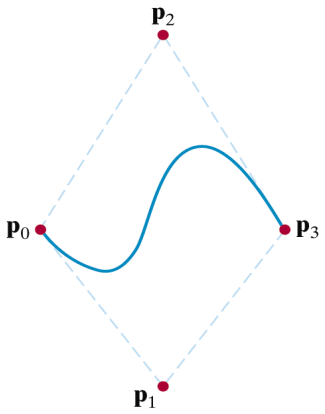
$$x'(0) = 3(x_1 - x_0), \qquad x'(1) = 3(x_3 - x_2)$$

Properties of Cubic Bezier Curve

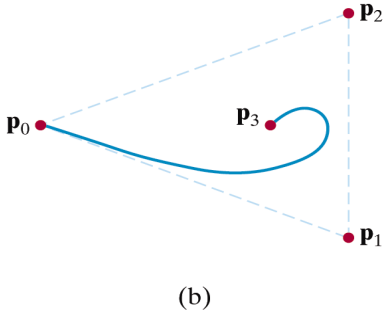
- The curve is contained in the convex hull of the control polygon

- Invariance under affine transformation
  - Applying affine transformation to the curve is the same as applying the transformation to the control points then evaluate the curve

$$\sum B_i(t) = 1$$
$$0 \le B_i(t) \le 1$$
$$\text{if } \ 0 \le t \le 1$$



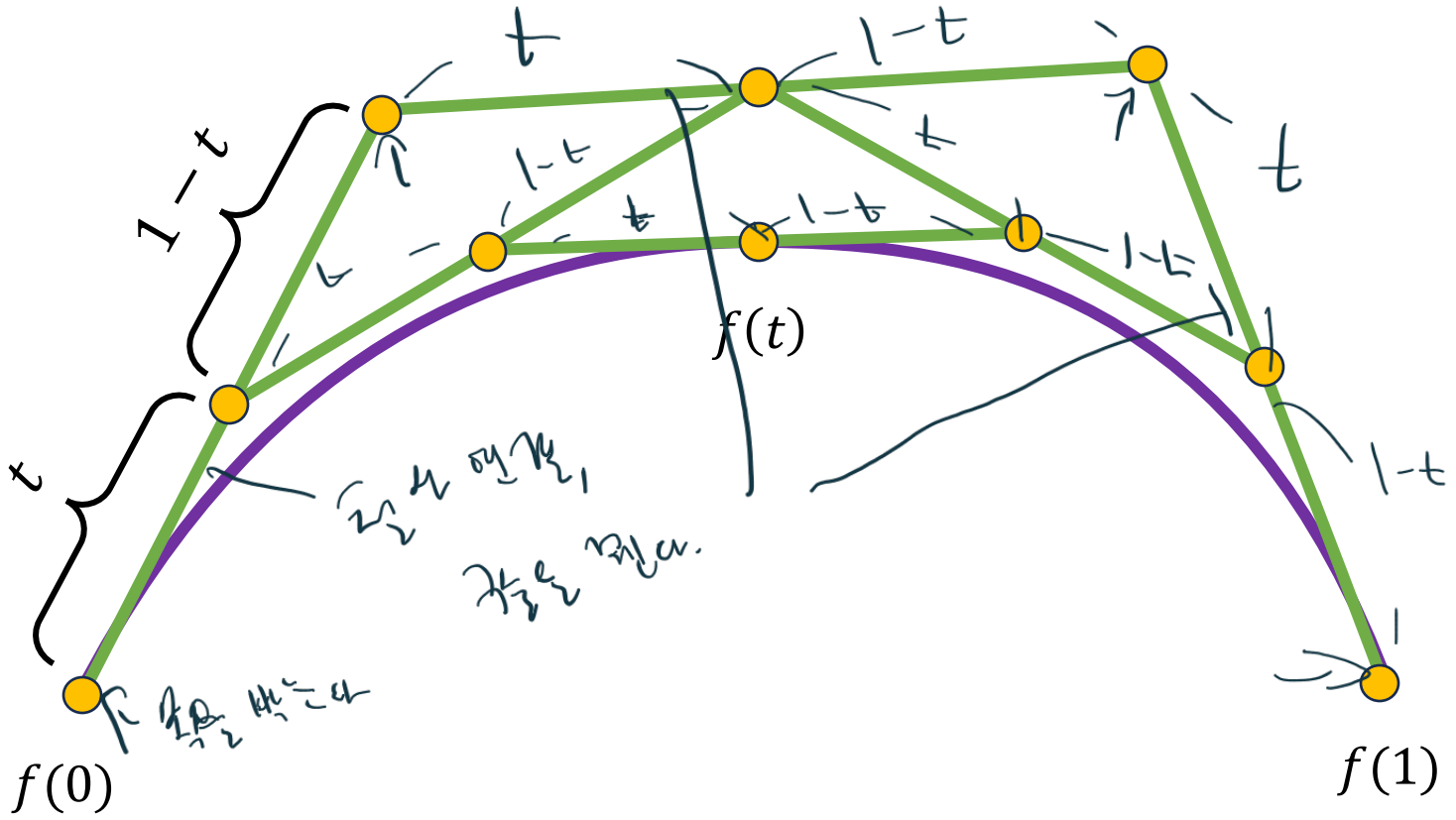
(a)



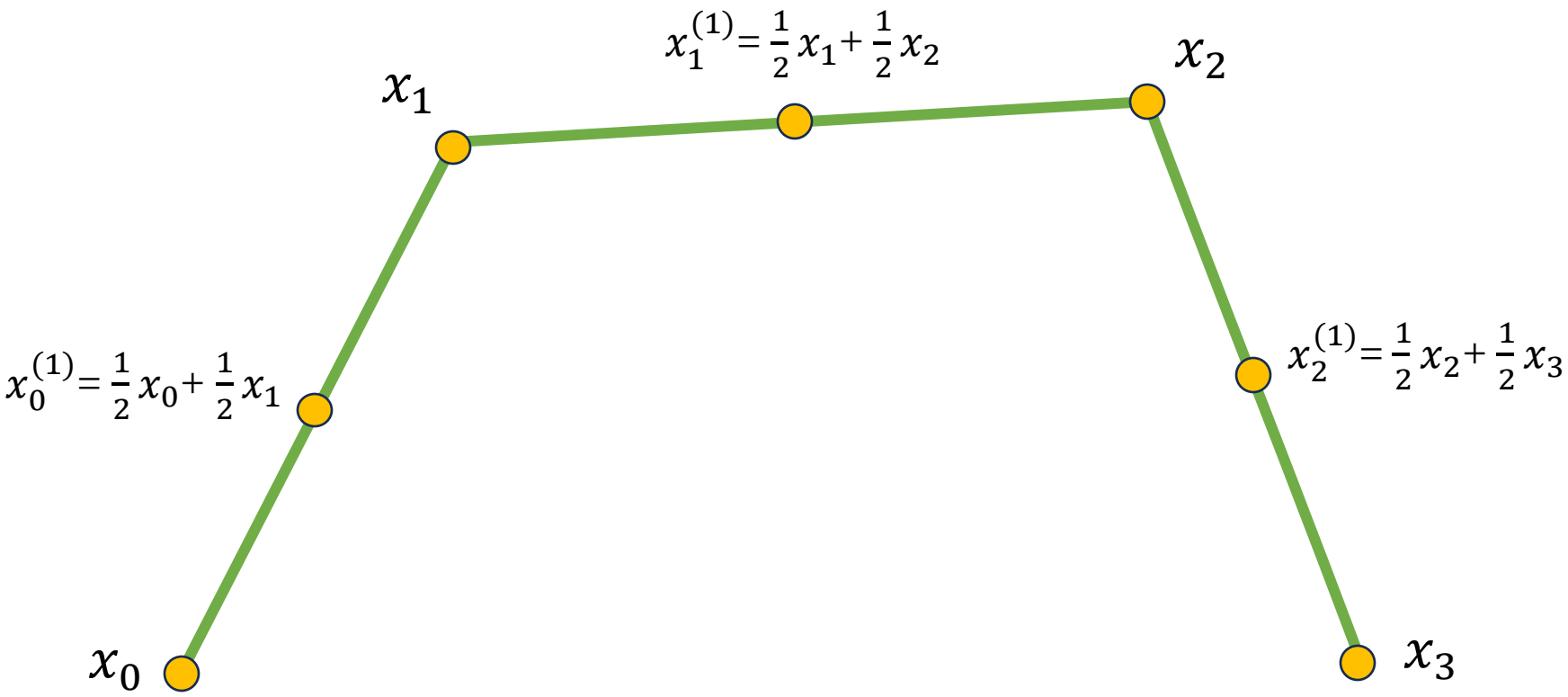
(b)

De Casteljau Algorithm

- $x(t)$ can be evaluated by recursively subdividing control polygon with the ratio of $(t, 1-t)$

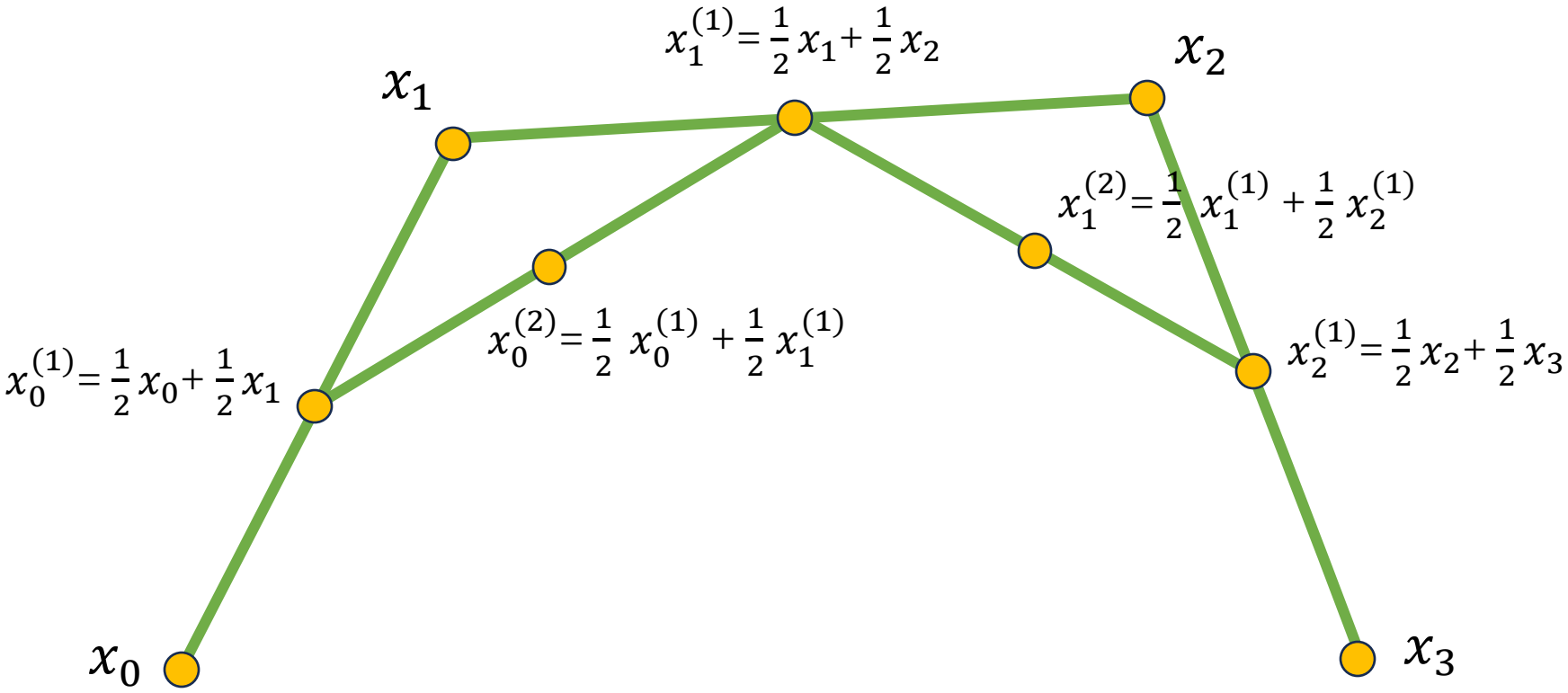


De Casteljau Algorithm



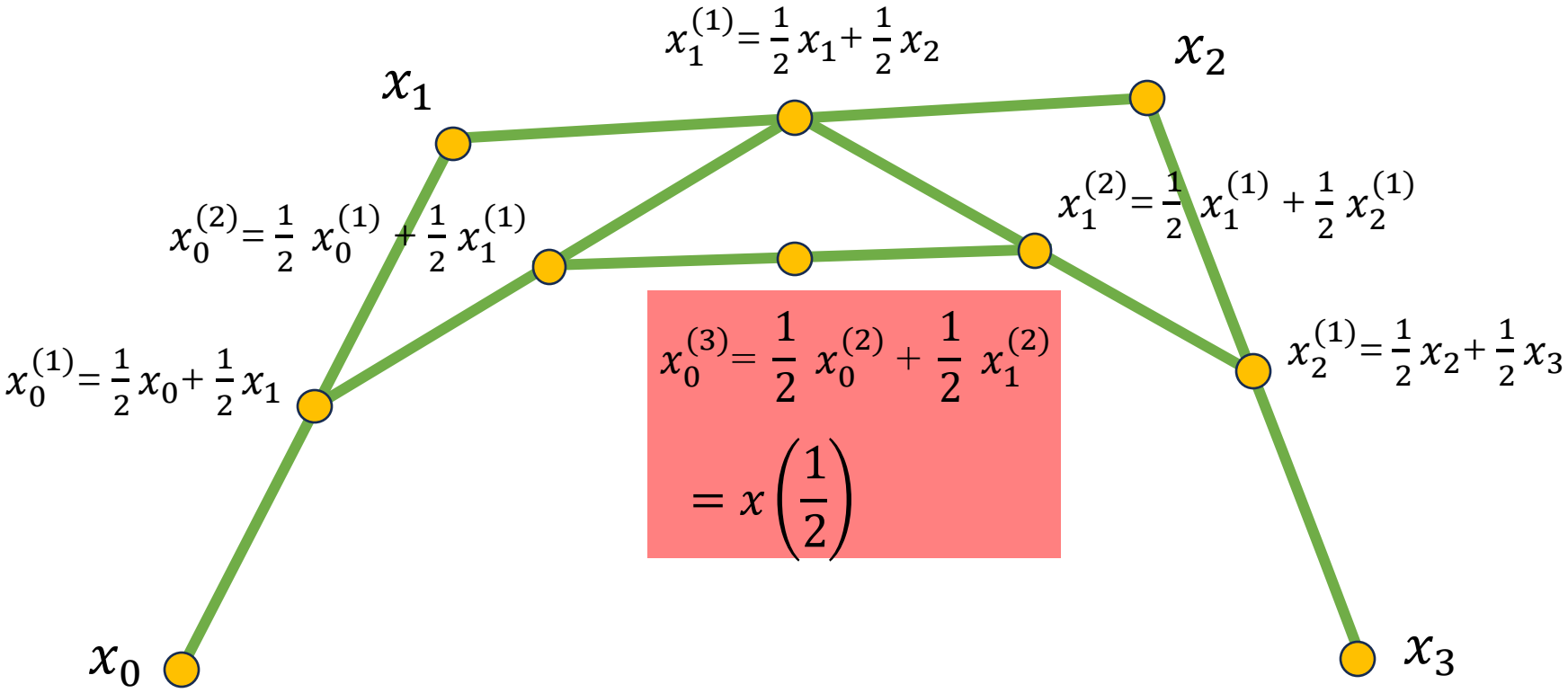
By symmetric and affine properties

De Casteljau Algorithm



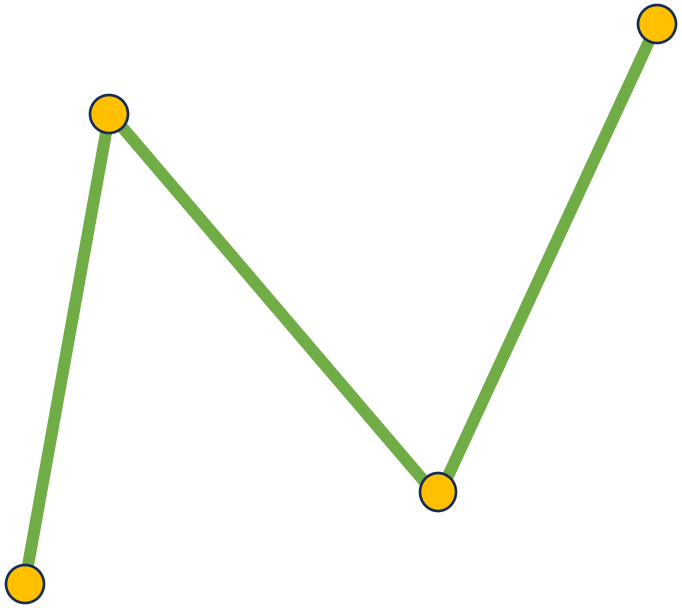
By affine properties

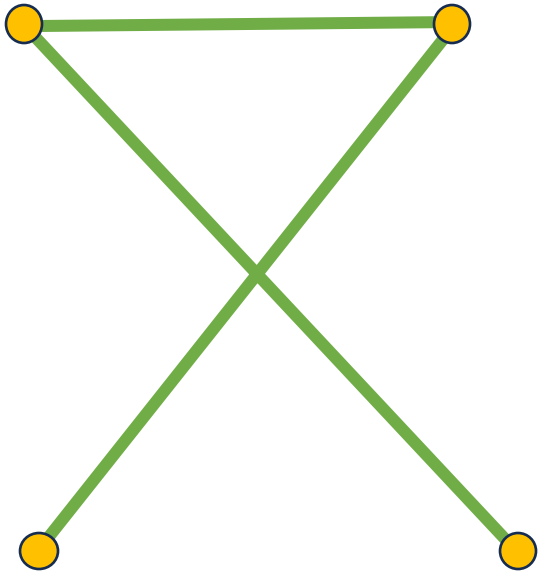
De Casteljau Algorithm



Exercise

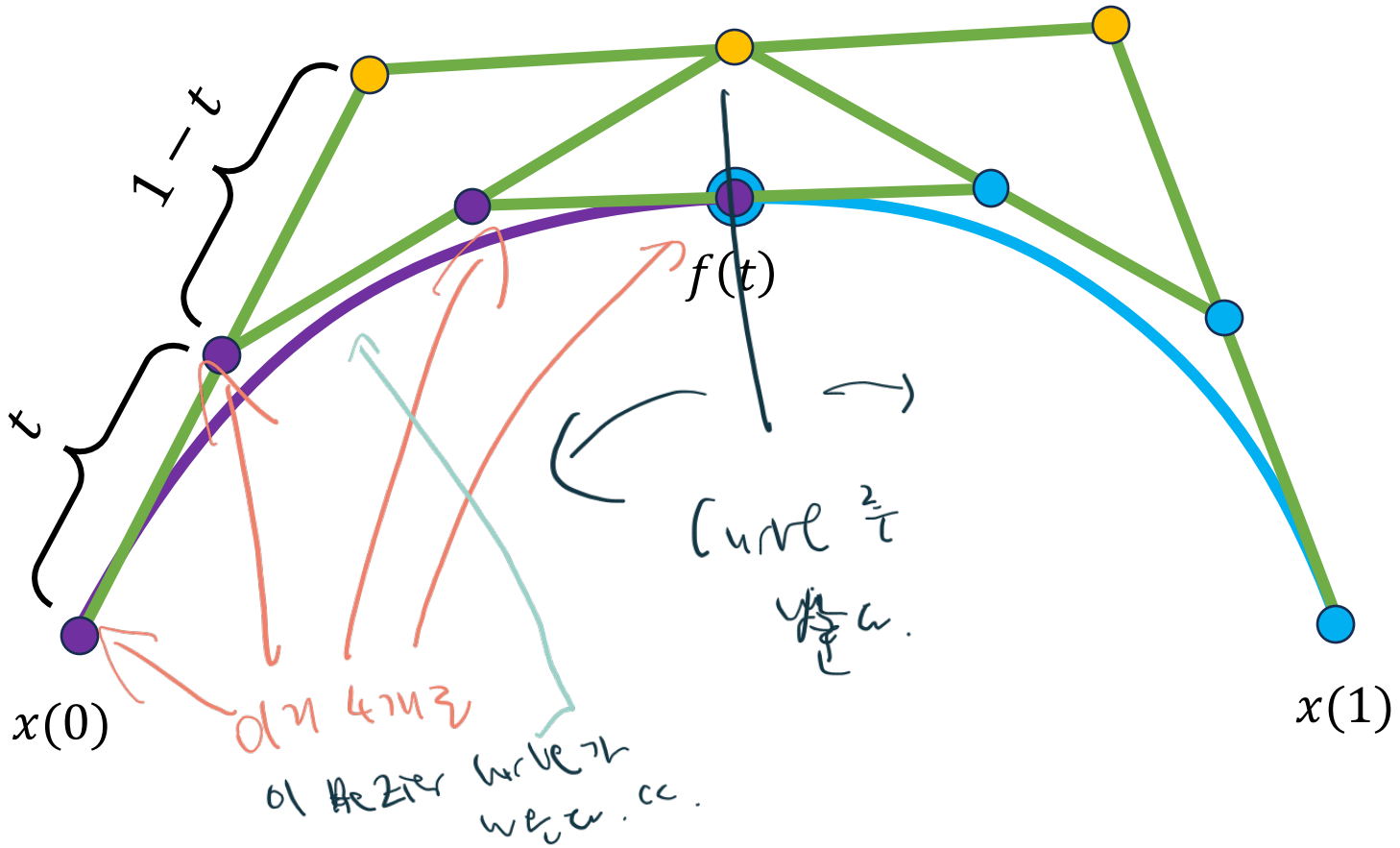
$$x\left(\frac{1}{4}\right), x\left(\frac{1}{2}\right), x\left(\frac{3}{4}\right)?$$





Extra Observation

- After the subdivision, $x(t)$ can be divided into two parts with new control polygons!



More than One Dimension

- We have discussed constructing a curve $x(t)$ existing in one dimension

- Curves in 2D, 3D or even higher dimensions can be constructed by simply expanding coordinates

$$P(t) = (x(t), y(t)) \qquad P(t) = (x(t), y(t), z(t))$$

- The tangent vector at t is simply a vector whose components are the derivatives of each 1D curve

$$P'(t) = (x'(t), y'(t)) \qquad P'(t) = (x'(t), y'(t), z'(t))$$

More than One Dimension

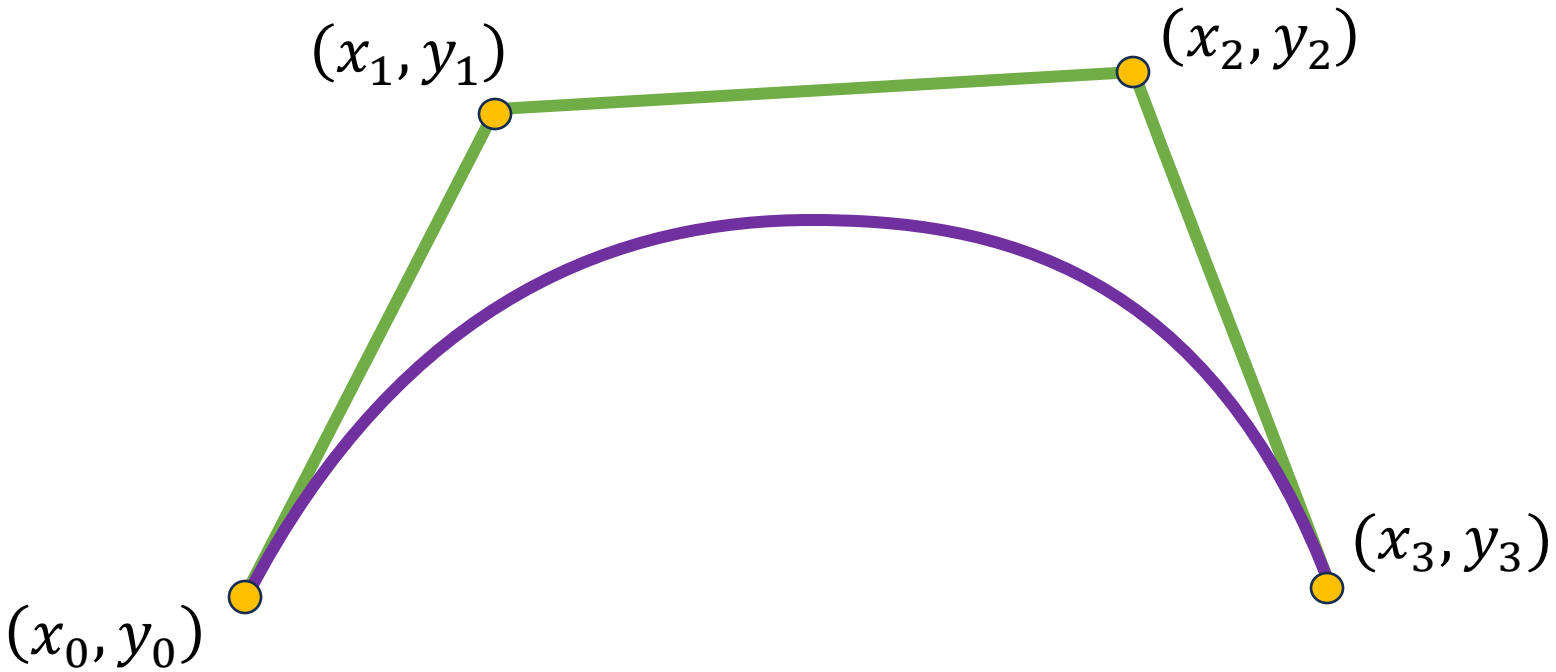
- One issue to think about:

$$f_1(t) = (t, 2t) \qquad f_2(t) = (t^3, 2t^3)$$

- Both $f_1(t)$ and $f_2(t)$ are equal to $y = 2x$ but their parameterizations are different. In general, there can exist many parameterizations to represent the same shapes

- This does not matter if we only care about geometry of the curve, but it matters if we also care about movement along the curves (i.e., animation)

2D Bezier Curve



$$P(t) = \big(x(t), y(t)\big)$$

$$x(t) = x_0(1-t)^3 + x_1 3t(1-t)^2 + x_2 3t^2(1-t) + x_3 t^3$$

$$y(t) = y_0(1-t)^3 + y_1 3t(1-t)^2 + y_2 3t^2(1-t) + y_3 t^3$$

2D Bezier Curve (Matrix Form)

$$P(t) = \big(x(t), y(t)\big)$$

$$x(t) = x_0(1-t)^3 + x_1 3t(1-t)^2 + x_2 3t^2(1-t) + x_3 t^3$$

$$y(t) = y_0(1-t)^3 + y_1 3t(1-t)^2 + y_2 3t^2(1-t) + y_3 t^3$$

$$P(t) = \begin{bmatrix} x_0 & x_1 & x_2 & x_3 \\ y_0 & y_1 & y_2 & y_3 \end{bmatrix} \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 0 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ t \\ t^2 \\ t^3 \end{bmatrix}$$

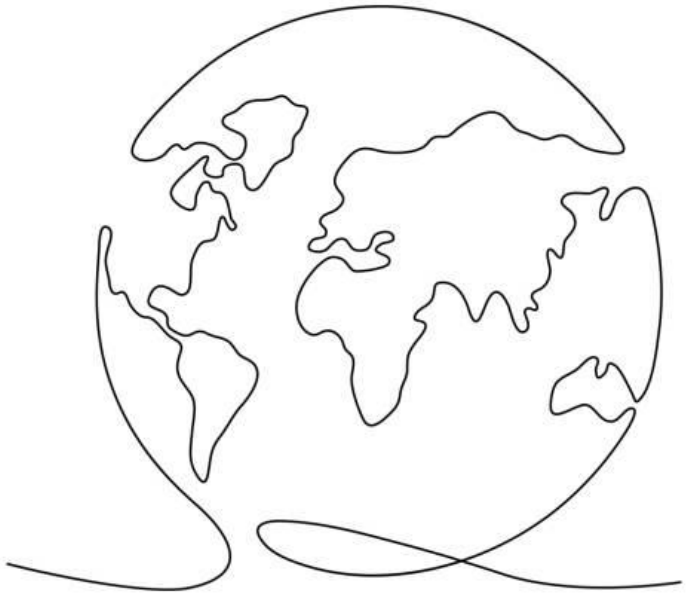
$$= [\boldsymbol{p}_0 \quad \boldsymbol{p}_1 \quad \boldsymbol{p}_2 \quad \boldsymbol{p}_3] \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 0 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ t \\ t^2 \\ t^3 \end{bmatrix}$$

Control point

Same extension for 3D Bezier curve

What About This Curve?

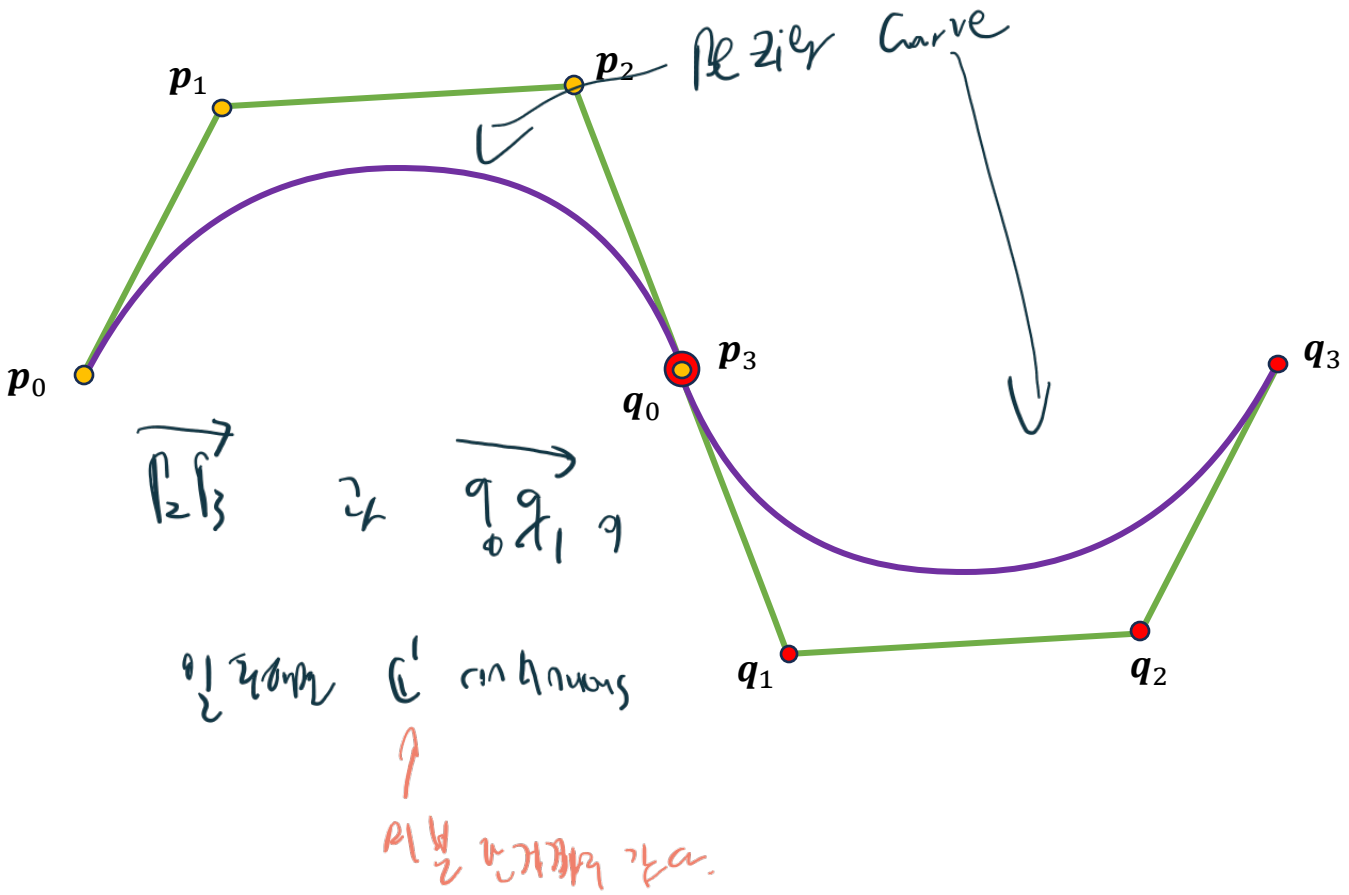
- One cubic is not enough to draw complex objects



⇒ Spline

Spline

- A piecewise polynomial with a high degree of ***smoothness*** where pieces meet



Continuity Conditions

- To ensure a smooth transition from one section of a piecewise parametric spline to the next, we can impose various continuity conditions at the connection points
- Parametric continuity
  - Matching the parametric derivatives of adjoining curve sections at their common boundary
- Geometric continuity
  - Geometric smoothness independent of parametrization

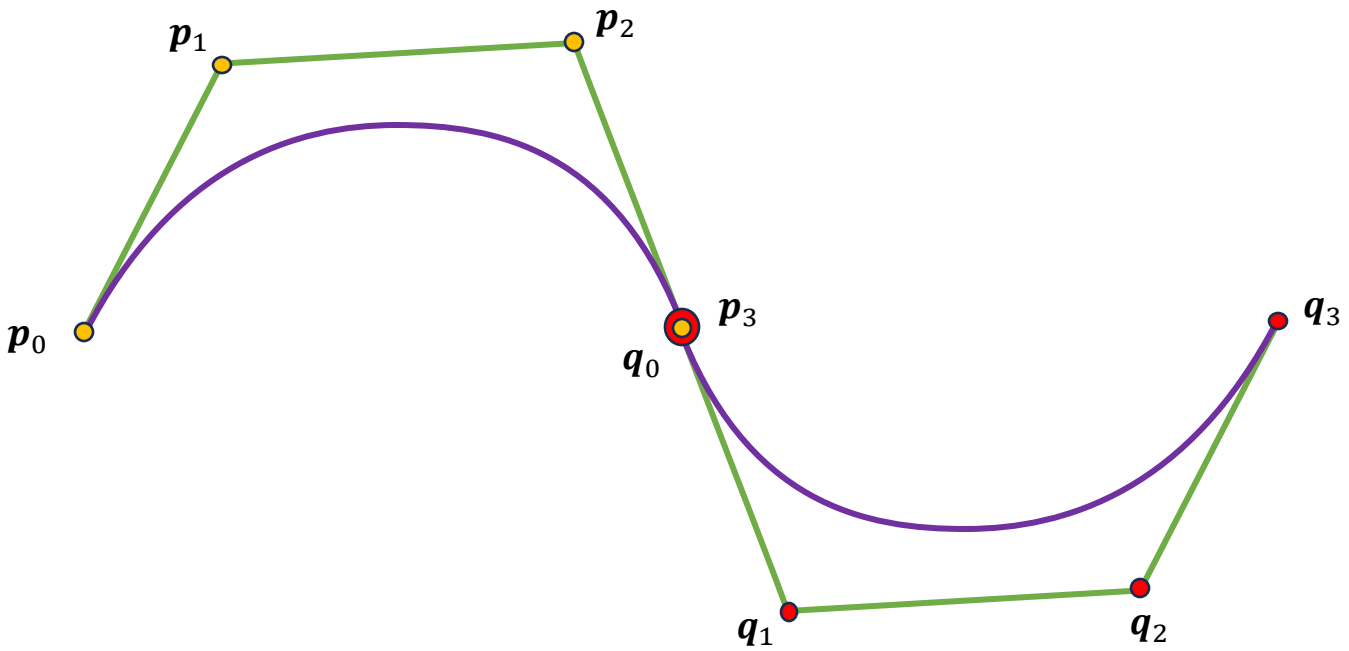
$$P(t) = (t^2, t^3)$$

$$P(t) = \begin{cases} (t, t^2) & \text{when } t < 0 \\ (t^2, t^4) & \text{when } t \geq 0 \end{cases}$$

Orders of Continuity

- C^0, G^0 continuity (positional continuity)
  - The positions of common control points are the same
- G^1 continuity (tangential continuity)
  - C^0 and the directions of derivatives are the same
- C^1 continuity
  - C^0 and the derivatives are the same
- G^2 continuity (curvature continuity)
  - G^1 and the curvatures are the same
- C^2 continuity
  - C^1 and the second derivatives are the same

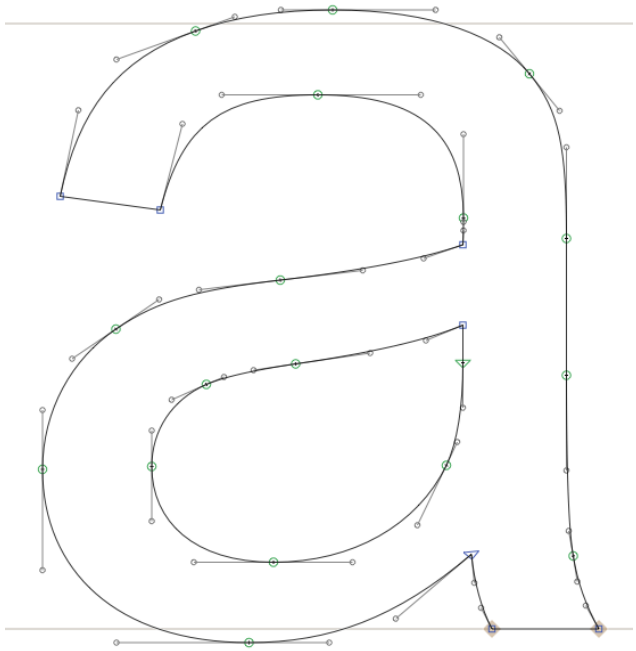
Connecting Cubic Bezier Curves



- How can we guarantee C^0 continuity? $p_3 = q_0$
- How can we guarantee G^1 continuity? p_2, p_3, q_0, q_1 가 한 직선
- How can we guarantee C^1 continuity? $G^1 + \overline{p_2 p_3} = \overline{q_0 q_1}$

Fonts





TrueType : Quadratic Bezier
PostScript : Cubic Bezier

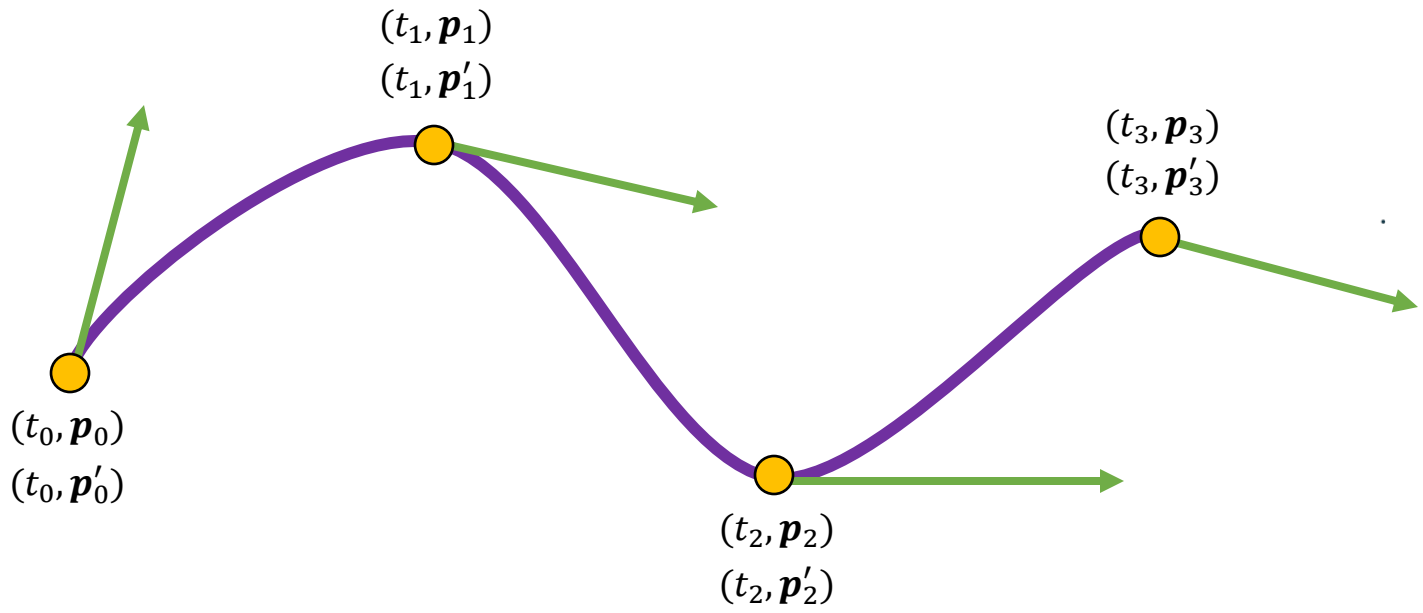
https://medium.com/specle/what-is-true-type-font-1a13d92fe19e

Bezier Curves: Drawback

- For every connections, three control points should be carefully aligned to meet tangent continuity, which would be burdensome for users

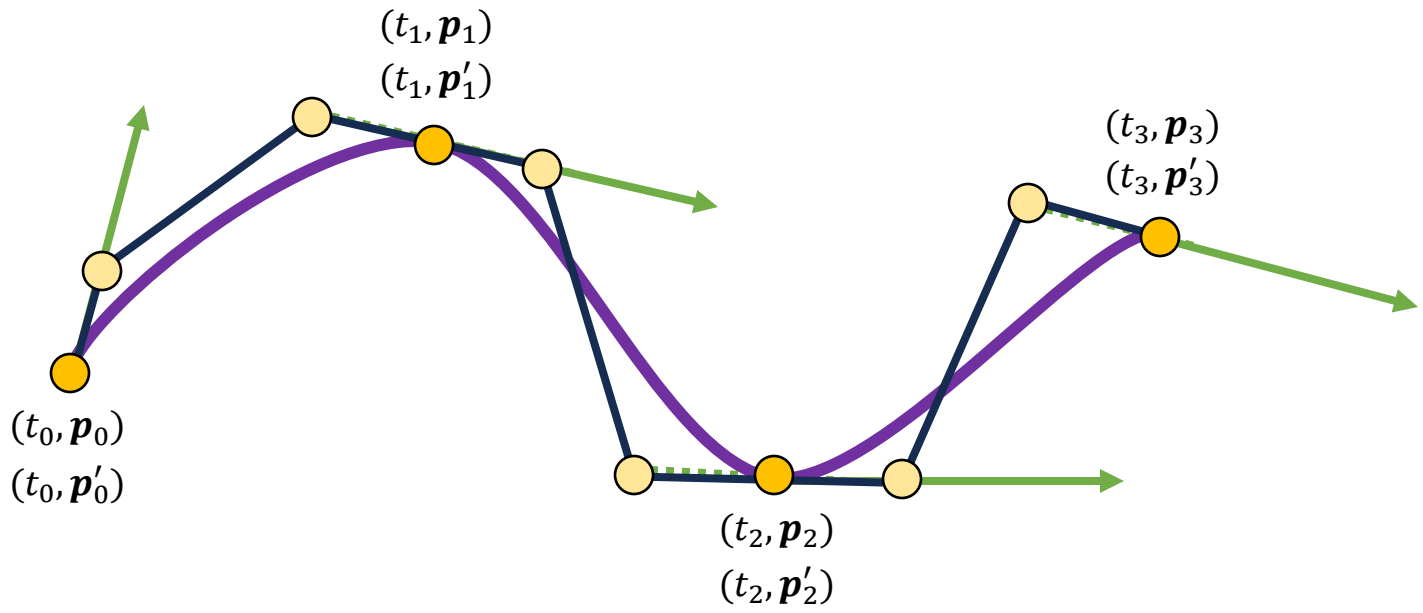
Bezier Splines with Tangent Conditions

- Find a piecewise Bezier curve that passes through given key frames and tangent vectors



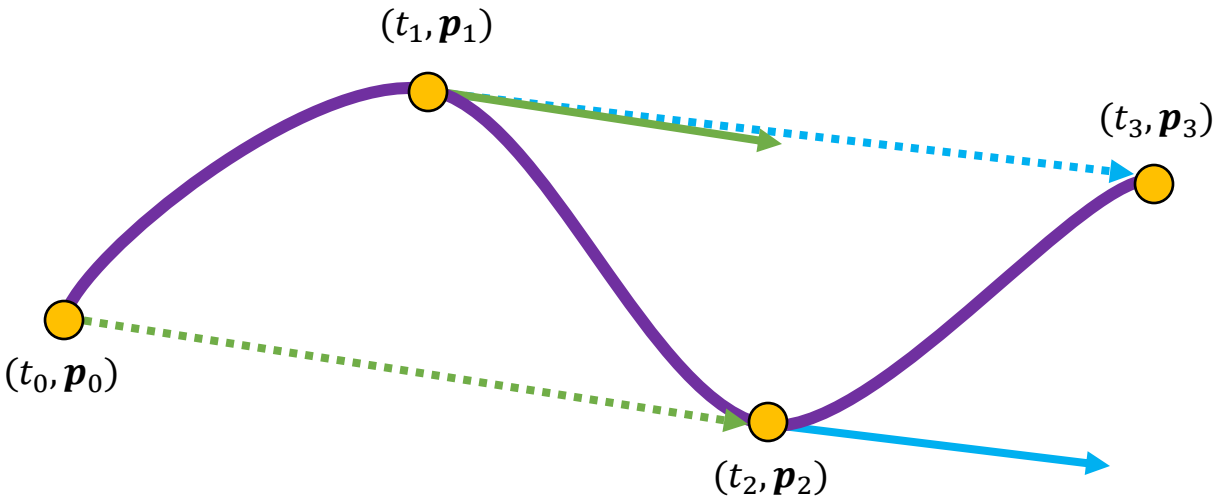
Bezier Splines with Tangent Conditions

- Find a piecewise Bezier curve that passes through given key frames and tangent vectors



Catmull-Rom Splines

- Tangents are estimated via adjacent points while the first and last tangents are set to be zero

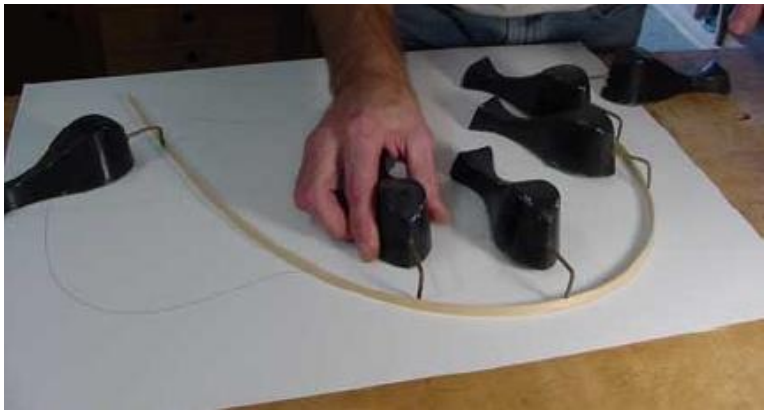


$$\boldsymbol{p}'_0 = \boldsymbol{p}'_n = \boldsymbol{0}, \qquad \boldsymbol{p}'_i = \frac{\boldsymbol{p}_{i+1} - \boldsymbol{p}_{i-1}}{2} \quad \text{for } 0 < i < n$$

Natural Cubic Splines

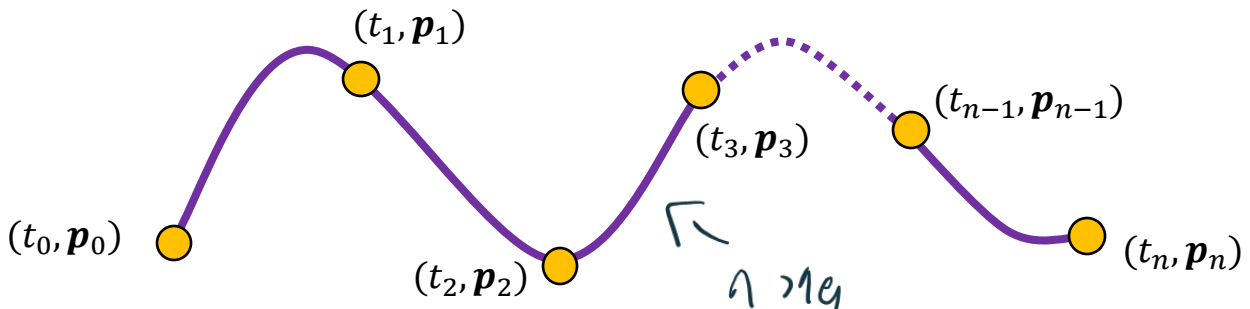
- Is it possible to achieve higher continuity?
  - Theoretically, C^{n-1} continuity can be achieved from splines of degree n

- Motivated by loftman's spline
  - Long narrow strip of wood or plastic
  - Shaped by lead weights (called ducks)



Natural Cubic Splines

- Given $n+1$ control points, we want to find n connected Bezier curves passing through the points with C^2 continuity



- For each dimension
  - We have 4n unknowns (4 control points per each curve)
  - We have (4n-2) equations
    - 2n equations for end point interpolations
    - (n-1) equations for tangential continuity
    - (n-1) equations for **second derivative continuity**

Two more equations are still required to solve!

Natural Cubic Splines

- Boundary conditions for open curve

$$\boldsymbol{p}''(t_0) = \boldsymbol{p}''(t_n) = 0$$

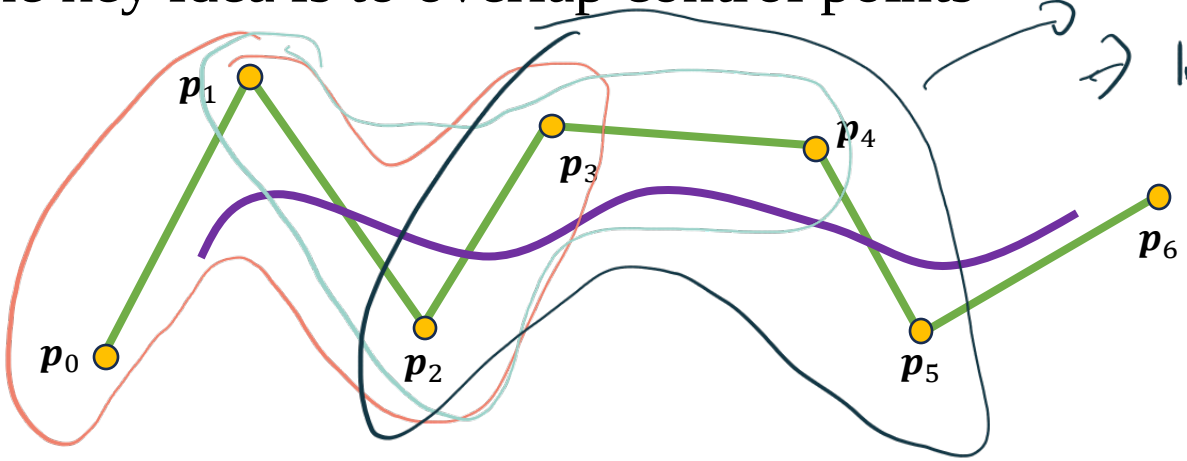
- Boundary conditions for closed curve

$$\boldsymbol{p}'(t_0) = \boldsymbol{p}'(t_n) \quad \text{and} \quad \boldsymbol{p}''(t_0) = \boldsymbol{p}''(t_n)$$

- Properties
  - High-continuity is achieved
  - The curve no longer has local controllability, meaning any change in one control point affects the entire shape of the curve

B-splines: Motivation

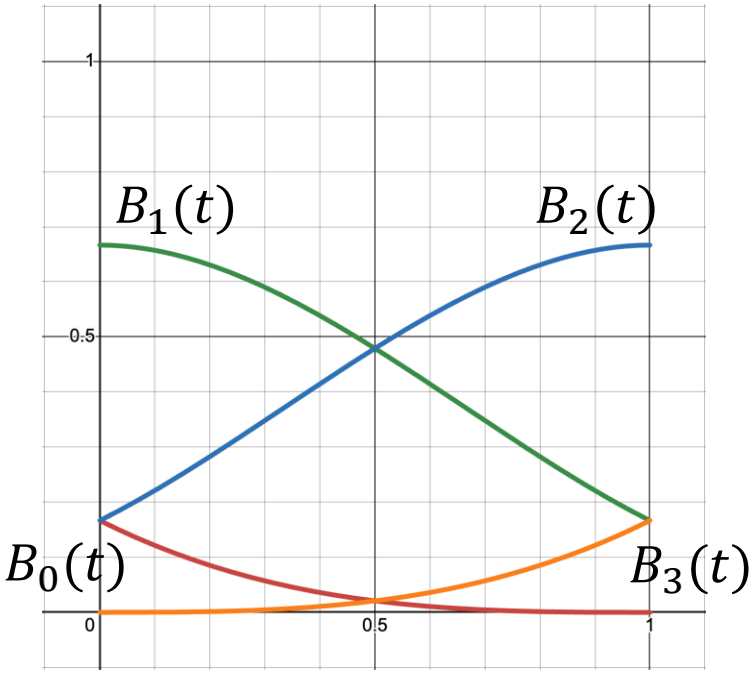
- Is it possible to achieve both high-order contunity and local controllability?
  - B-splines can do!
  - The key idea is to overlap control points



- But, curves is no longer constrained to pass through any control points

basis or splines

Uniform Cubic B-spline bases

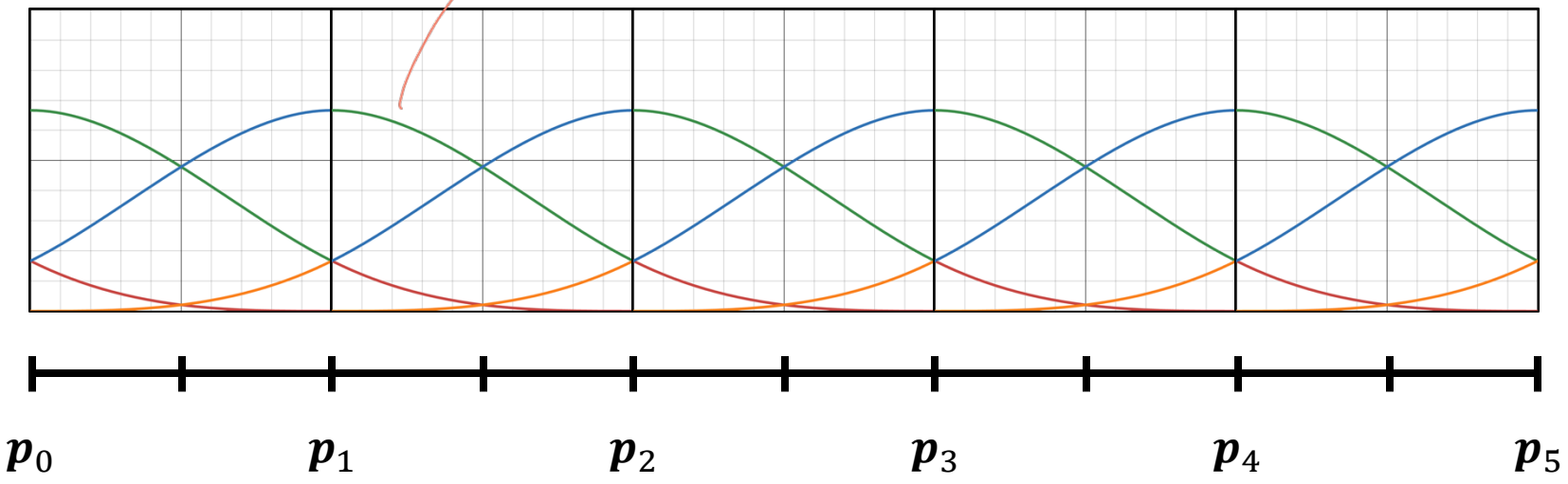


$$B_0(t) = \frac{1}{6}(1-t)^3$$
$$B_1(t) = \frac{1}{6}(3t^3 - 6t^2 + 4)$$
$$B_2(t) = \frac{1}{6}(-3t^3 + 3t^2 + 3t + 1)$$
$$B_3(t) = \frac{1}{6}t^3$$

$$\boldsymbol{p}(t) = \boldsymbol{p}_0 B_0(t) + \boldsymbol{p}_1 B_1(t) + \boldsymbol{p}_2 B_2(t) + \boldsymbol{p}_3 B_3(t)$$

Uniform Cubic B-spline bases

- A bell-shaped basis function for each control points
- Overlapping basis functions



Uniform Cubic B-spline (Matrix Form)

$$\boldsymbol{p}(t) = \boldsymbol{p}_0 B_0(t) + \boldsymbol{p}_1 B_1(t) + \boldsymbol{p}_2 B_2(t) + \boldsymbol{p}_3 B_3(t)$$

Carl de Boor

where

$$B_0(t) = \frac{1}{6}(1-t)^3$$
$$B_1(t) = \frac{1}{6}(3t^3 - 6t^2 + 4)$$
$$B_2(t) = \frac{1}{6}(-3t^3 + 3t^2 + 3t + 1)$$
$$B_3(t) = \frac{1}{6}t^3$$

$$\boldsymbol{p}(t) = [\boldsymbol{p}_0 \quad \boldsymbol{p}_1 \quad \boldsymbol{p}_2 \quad \boldsymbol{p}_3] \frac{1}{6} \begin{bmatrix} 1 & -3 & 3 & -1 \\ 4 & 0 & -6 & 3 \\ 1 & 3 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ t \\ t^2 \\ t^3 \end{bmatrix}$$

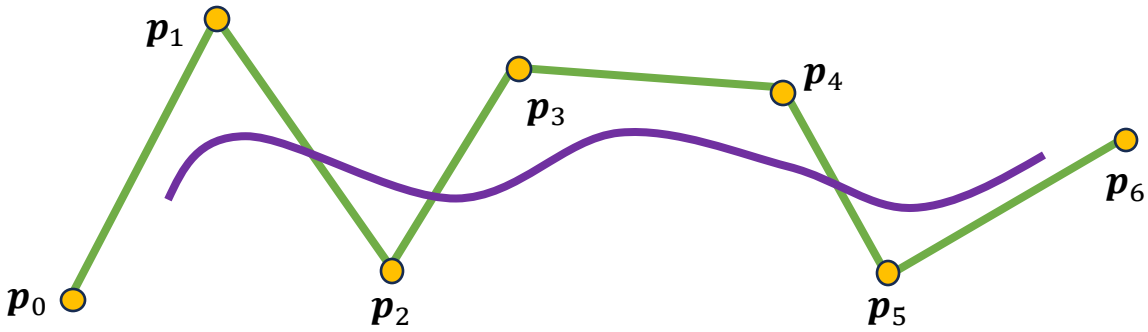
Geometry (control points) **G**

Change-of-basis **B**

Monomial basis **M(t)**

B-Splines Properties

- Convex hull
- Affine invariance
- Variation diminishing
- C^{n-1} continuity
- Local controllabilty



Conversion between Bezier & B-Spline

- Sometimes it is desirable to be able to switch from one spline representation to another
- Simple with the basis matrices

$$\begin{aligned}\boldsymbol{p}(t) &= \mathbf{G}_{Bezier} \cdot \mathbf{B}_{Bezier} \cdot \mathbf{M}(\mathrm{t}) \\ &= \left(\mathbf{G}_{Bezier} \cdot \mathbf{B}_{Bezier} \cdot \mathbf{B}_{Bspline}^{-1}\right) \cdot \mathbf{B}_{Bspline} \cdot \mathbf{M}(\mathrm{t}) \\ &= \mathbf{G}_{Bspline} \cdot \mathbf{B}_{Bspline} \cdot \mathbf{M}(\mathrm{t})\end{aligned}$$

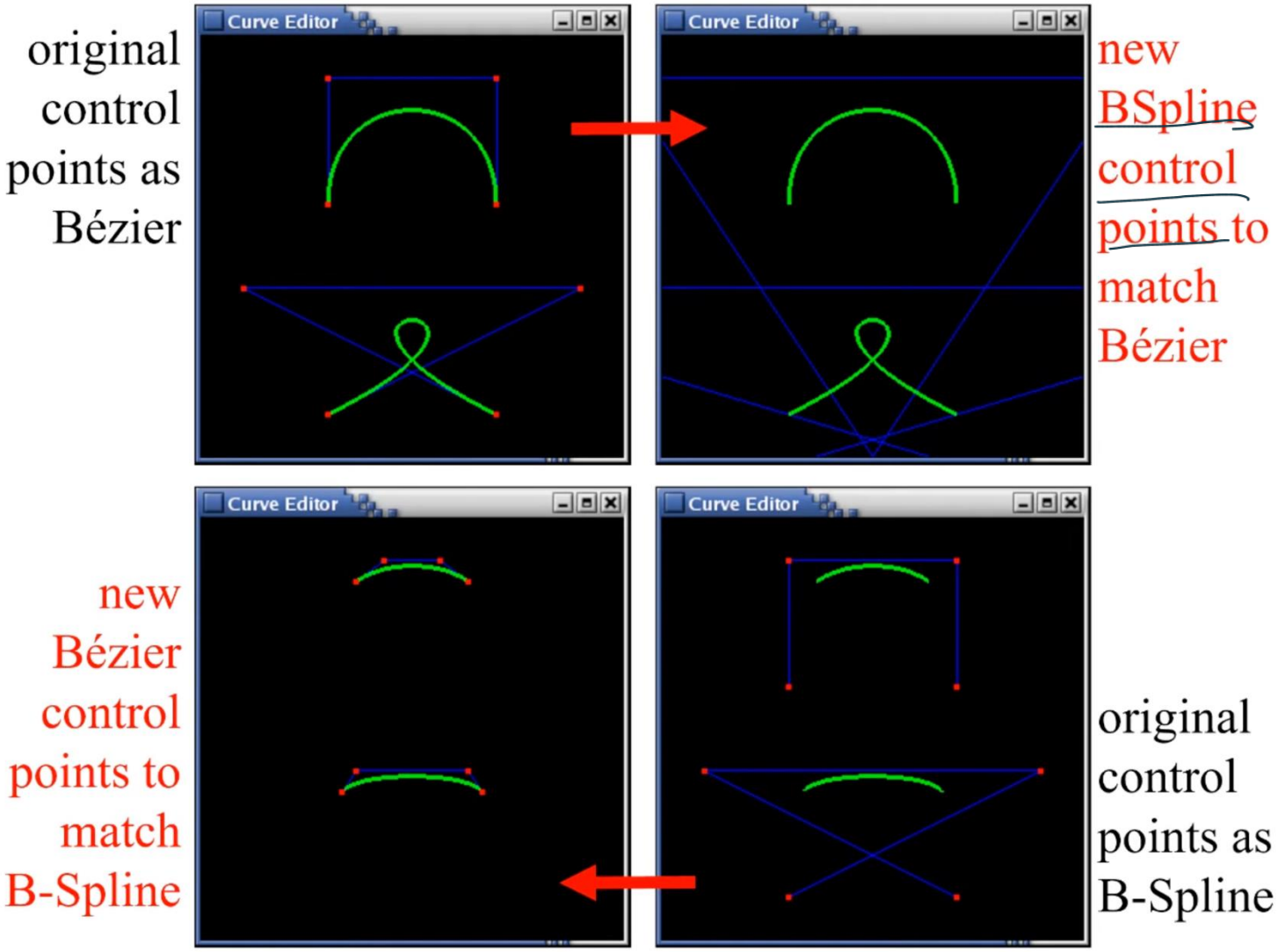
$$B_{Bezier} = \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 0 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where

$$B_{Bspline} = \frac{1}{6}\begin{bmatrix} 1 & -3 & 3 & -1 \\ 4 & 0 & -6 & 3 \\ 1 & 3 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Note that this only works for a single segment of 4 control points

Conversion between Bezier & B-Spline



NURBS (Generalized B-Splines)

- Non-Uniform Rational B-Spline
  - Rational polynomial
    - A polynomial divided by a polynomial
    - Can represent conics (circle, ellipses and hyperbolics)
    - Invariant under projective transformation
    - Implemented by adding the homogeneous coordinate ω into the control points
  - Non-uniform
    - Different spacing between the blending functions (a.k.a. knots)

$$\boldsymbol{p}(t) = \frac{\sum \omega_j \boldsymbol{p}_j B_j(t)}{\sum \omega_j B_j(t)}$$

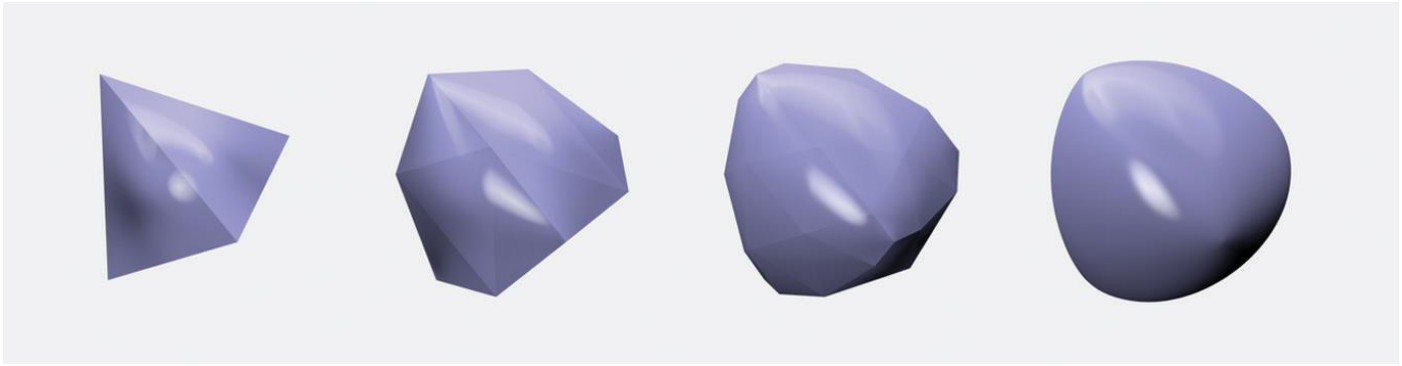
- Side-note
  - Uniform B-Spline is a special case of non-uniform B-Spline
  - Non-rational B-Spline is a special case of rational B-Spline

From Curves to Surfaces

- Tensor Product Splines
  - Surface analog of spline curves



- Subdivision surfaces

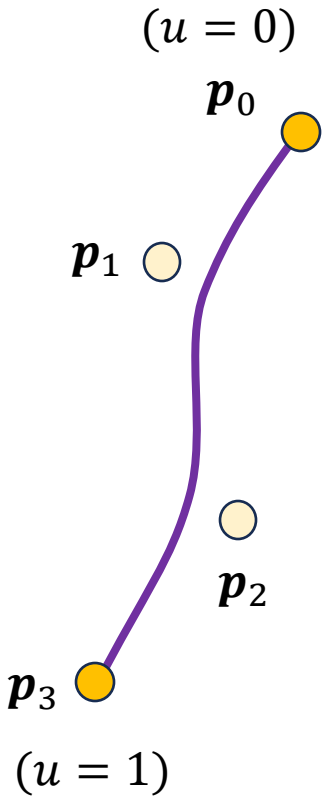


Tensor Product Surfaces

Bernstein Basis

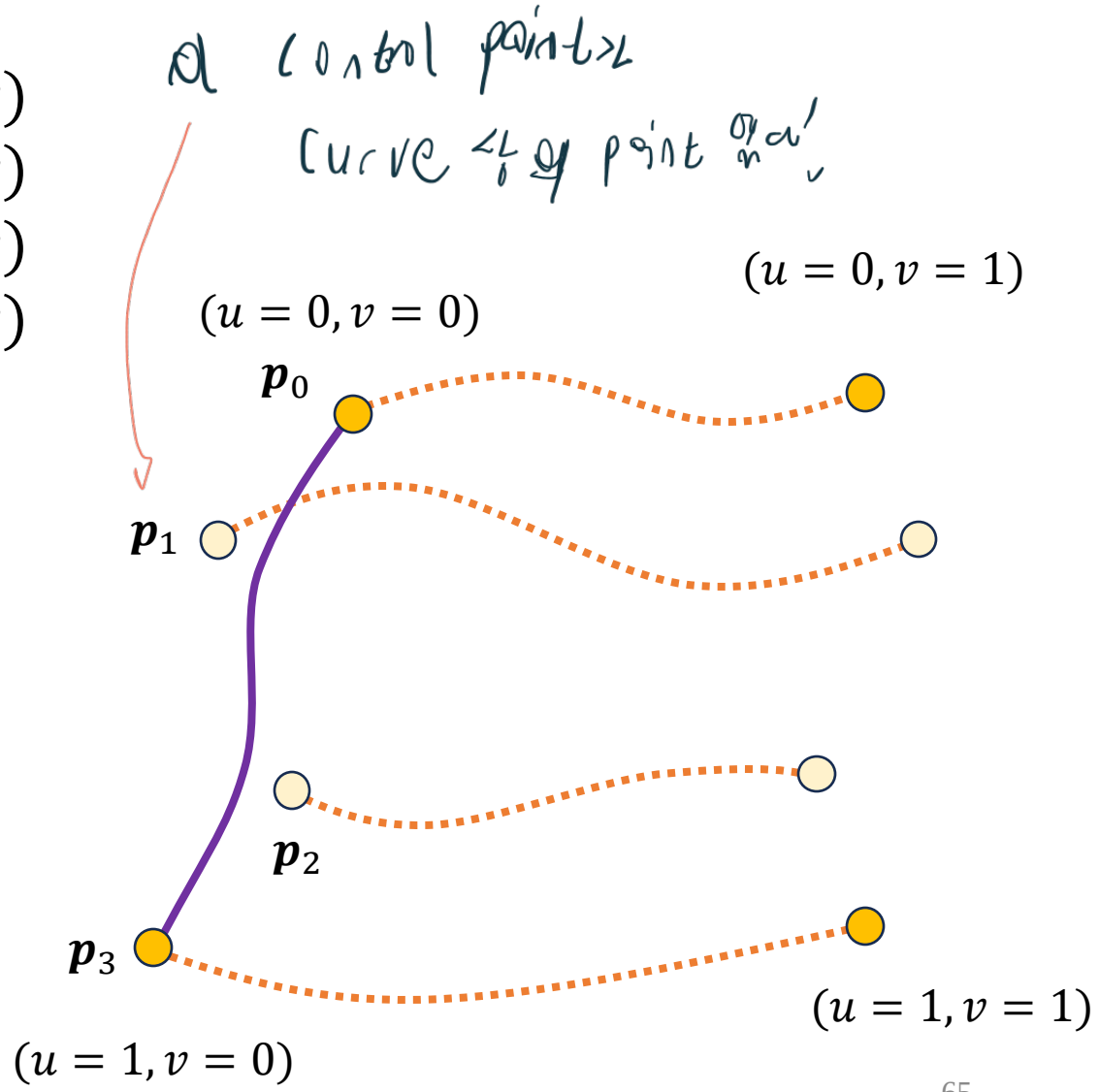
$$\boldsymbol{p}(u) = (1-u)^3\boldsymbol{p}_0 + 3u(1-u)^2\boldsymbol{p}_1 + 3u^2(1-u)\boldsymbol{p}_2 + u^3\boldsymbol{p}_3$$

(Note: relabeled t to u)



Tensor Product Surfaces

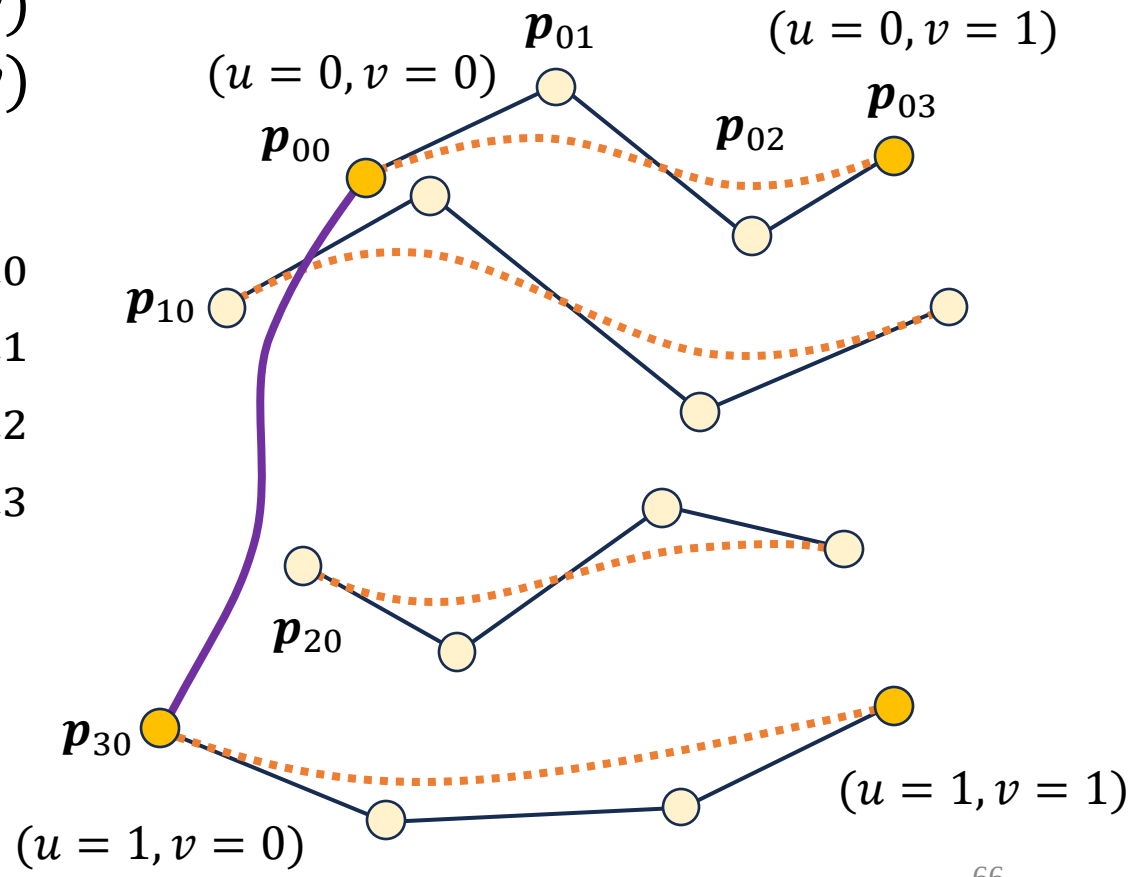
$$\boldsymbol{p}(u,v) = (1-u)^3\boldsymbol{p}_0(v) + 3u(1-u)^2\boldsymbol{p}_1(v) + 3u^2(1-u)\boldsymbol{p}_2(v) + u^3\boldsymbol{p}_3(v)$$



Bicubic Bezier Surfaces

$$\boldsymbol{p}(u,v) = (1-u)^3\boldsymbol{p}_0(v) + 3u(1-u)^2\boldsymbol{p}_1(v) + 3u^2(1-u)\boldsymbol{p}_2(v) + u^3\boldsymbol{p}_3(v)$$

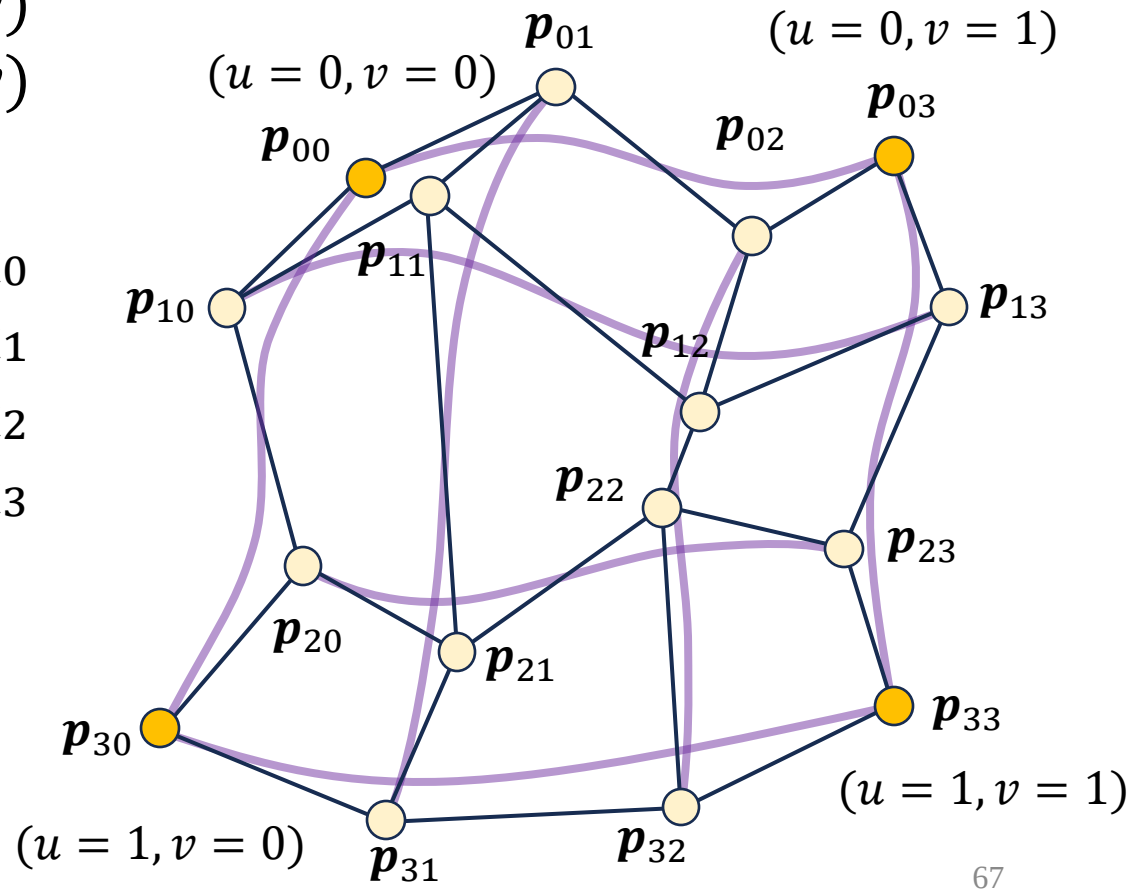
$$\boldsymbol{p}_i(v) = (1-v)^3\boldsymbol{p}_{i0} + 3v(1-v)^2\boldsymbol{p}_{i1} + 3v^2(1-v)\boldsymbol{p}_{i2} + v^3\boldsymbol{p}_{i3}$$



Bicubic Bezier Surfaces

$$\boldsymbol{p}(u,v) = (1-u)^3\boldsymbol{p}_0(v) + 3u(1-u)^2\boldsymbol{p}_1(v) + 3u^2(1-u)\boldsymbol{p}_2(v) + u^3\boldsymbol{p}_3(v)$$

$$\boldsymbol{p}_i(v) = (1-v)^3\boldsymbol{p}_{i0} + 3v(1-v)^2\boldsymbol{p}_{i1} + 3v^2(1-v)\boldsymbol{p}_{i2} + v^3\boldsymbol{p}_{i3}$$



Bezier Surface Examples

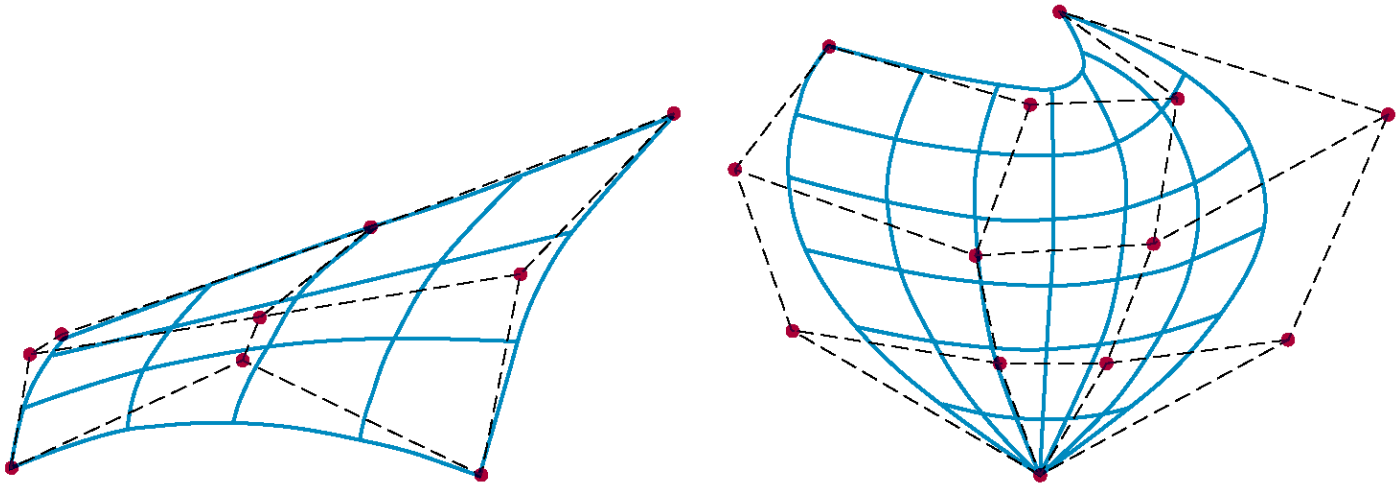


Figure 8-39

Wire-frame Bézier surfaces constructed with (a) nine control points arranged in a 3 by 3 mesh and (b) sixteen control points arranged in a 4 by 4 mesh. Dashed lines connect the control points.

Bicubics, Tensor Product

$$\boldsymbol{p}(u,v) = B_0(u)\boldsymbol{p}_0(v) + B_1(u)\boldsymbol{p}_1(v) + B_2(u)\boldsymbol{p}_2(v) + B_3(u)\boldsymbol{p}_3(v)$$

$$\boldsymbol{p}_i(v) = B_0(v)\boldsymbol{p}_{i0} + B_1(v)\boldsymbol{p}_{i1} + B_2(v)\boldsymbol{p}_{i2} + B_3(v)\boldsymbol{p}_{i3}$$

16 control points $\boldsymbol{p}_{ij}$

16 2D basis functions B_{ij}

$$\boldsymbol{p}(u,v) = \sum_{i=0}^{3} B_i(u) \left[\sum_{j=0}^{3} \boldsymbol{p}_{ij} B_j(v) \right]$$

$$= \sum_{i=0}^{3} \sum_{j=0}^{3} \boldsymbol{p}_{ij} B_{ij}(u,v)$$

where $B_{ij}(u,v) = B_i(u)B_j(v)$

Bicubic Bezier Surface (Matrix Form)

$$x(u,v) = \sum_{i=0}^{3}\sum_{j=0}^{3} x_{ij} B_i(u) B_j(v)$$

$$= \mathrm{M(u)}^{\mathrm{T}} \cdot \mathrm{B}_{Bezier}^{\mathrm{T}} \cdot \mathrm{G}^{x} \cdot \mathrm{B}_{Bezier} \cdot \mathrm{M}(v)$$

$$= \begin{bmatrix} 1 & u & u^2 & u^3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ -3 & 3 & 0 & 0 \\ 3 & -6 & 3 & 0 \\ -1 & 3 & -3 & 1 \end{bmatrix} \begin{bmatrix} x_{00} & x_{01} & x_{02} & x_{03} \\ x_{10} & x_{11} & x_{12} & x_{13} \\ x_{20} & x_{21} & x_{22} & x_{23} \\ x_{30} & x_{31} & x_{32} & x_{33} \end{bmatrix} \begin{bmatrix} 1 & -3 & 3 & -1 \\ 0 & 3 & -6 & 3 \\ 0 & 0 & 3 & -3 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ v \\ v^2 \\ v^3 \end{bmatrix}$$

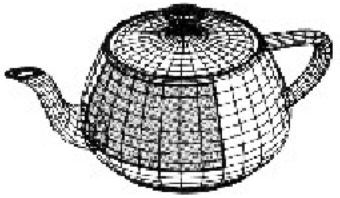
$$y(u,v) = \mathrm{M(u)}^{\mathrm{T}} \cdot \mathrm{B}_{Bezier}^{\mathrm{T}} \cdot \mathrm{G}^{y} \cdot \mathrm{B}_{Bezier} \cdot \mathrm{M}(v)$$

$$z(u,v) = \mathrm{M(u)}^{\mathrm{T}} \cdot \mathrm{B}_{Bezier}^{\mathrm{T}} \cdot \mathrm{G}^{z} \cdot \mathrm{B}_{Bezier} \cdot \mathrm{M}(v)$$

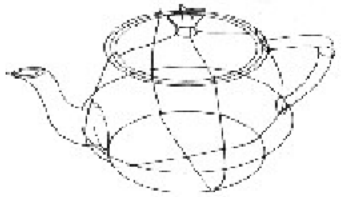
Bezier Patches Examples: Utah Teapot

- The first 3D model desinged using bezier curves, consisting of 32 (designed by Martin Newell)

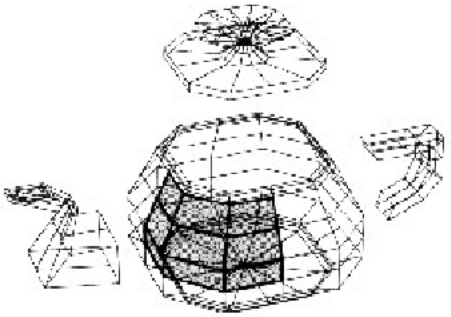




single shaded patch



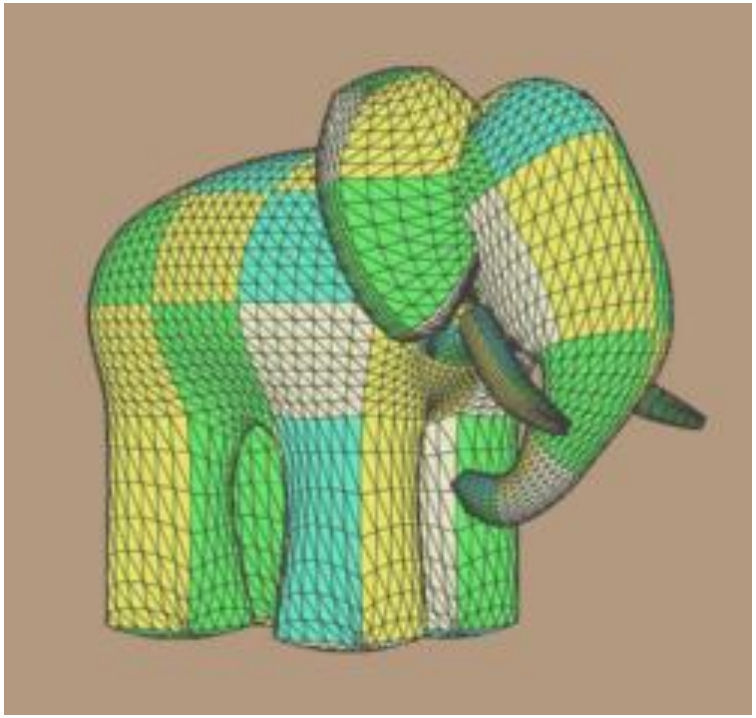
Patch edges



wireframe of the control points

- Using a teapot model is considered the 3D equivalent of a "Hello, World!" program in Computer Graphics programming
- Some programming libraries, such as the OpenGL Utility Toolkit even have functions dedicated to drawing teapots

Bezier Patches Examples: Gumbo



Ed Catmull's "Gumbo" model, composed of patches

https://en.wikipedia.org/wiki/B%C3%A9zier_surface

Tensor Product B-Spline Patches

- Bezier and B-Spline curves are both cubics
  - interchangeable through matrix transformations
- We can build tensor product surface patches out of B-Splines just as well
  - Still 4x4 control points / patch
  - 2D basis functions are products of B-Spline basis functions

Tensor Product Spline Patches

- Pros
  - Smooth
  - Defined by reasonably small set of points
- Cons
  - Hard to render (usually converted to triangles)
  - Tricky to ensure continuity at patch boundaries
- Extension
  - Rational splines: Splines in homogeneous coordinates
  - NURBS: Non-Uniform Rational B-Splines
    - Like curves: ratio of polynomials, non-uniform location of control points

Any Question?